

Elastispec: Formalizing Enterprise Firewall Management with Informal and Elastic Specifications

Chenan Wen, Yizhan Qing, Curt P Jansen, Xiaokang Qiu, and Sanjay Rao

Purdue University

West Lafayette, USA

{wen163, qing0, cjansen, xkqiu, sanjay}@purdue.edu

Abstract

Managing enterprise network firewalls is an ad-hoc process today, where administrators must extract policies relevant to their enterprises from thousands of natural language vendor documents and tailor them to their unique context. In this paper, we present Elastispec, a first step towards principled management of enterprise firewall policies with informal and incomplete specifications. We make three contributions: (i) LLM-assisted formalization of vendor documents into a custom domain specific language that precisely captures the rich choices and options using a multi-step consistency-preserving LLM agent; (ii) mapping the DSL to a concrete network environment by correlating diverse and possibly imperfect enterprise data sources; and (iii) an interactive auditor that cross-checks firewall configurations against the formal but potentially partial specifications and reports potential compliance gaps along with conjectures for human validation. Evaluations with real-world enterprise firewall configurations and popular enterprise application vendor documents show that Elastispec is effective in enabling operators to audit their configurations against vendor documents by producing compliance trees, enabling comparative analysis across parallel application deployments, and detecting configuration errors that permit non-compliant traffic.

CCS Concepts

• **Networks** → **Network management**; • **Security and privacy** → **Network security**.

Keywords

enterprise firewalls, network verification, domain-specific languages, LLM agents, configuration auditing

ACM Reference Format:

Chenan Wen, Yizhan Qing, Curt P Jansen, Xiaokang Qiu, and Sanjay Rao. 2026. Elastispec: Formalizing Enterprise Firewall Management with Informal and Elastic Specifications. In *ACM SIGCOMM 2026 Conference (SIGCOMM '26)*, August 17–21, 2026, Denver, CO, USA. ACM, Denver, CO, USA, 20 pages. <https://doi.org/10.1145/3789240.3829191>

1 Introduction

Enterprise networks host hundreds to thousands of applications (e.g., print management, backup, etc.) from diverse vendors. In managing their networks, enterprise operators must simultaneously

balance the twin goals of supporting the complicated communication patterns of these applications, while ensuring secure network operations. To this end, a key task is configuring dozens of enterprise firewalls so that only necessary traffic is permitted.

The last decade has seen tremendous progress on formal approaches to managing network configurations [1, 2, 4, 7, 9–11, 13, 16, 23, 24, 28–30, 42, 43, 46, 55–58, 61–63, 65, 67, 71, 76–79]. While laudable, a key challenge in bringing such techniques to bear is the need for precise and complete specifications. For instance, given specifications that indicate which pairs of IP addresses should (or should not) be allowed to communicate, existing network verification tools [2, 4, 9, 28–30, 42, 43, 55, 62, 65, 76–78] certify whether configurations match specifications. Unfortunately, such specifications are rarely available in enterprises today, owing to which the overall process of managing configurations remains ad-hoc, manual-intensive and prone to error.

Many factors contribute to the difficulty of obtaining precise specifications in enterprise environments. First, the communication patterns of applications are documented in hundreds of natural language vendor documents that are often ambiguous. Second, the documents cover a broad range of scenarios, only some of which may be relevant to an enterprise. For instance, they may describe policies for tens of application features and deployment options, databases, and authentication services not all of which may be deployed in a specific enterprise. Information on which features are relevant may not be formally documented, and is often spread across multiple operator teams. Third, the distributed nature of enterprises (multiple campuses and departments) may lead to multiple deployments of the same application that could each be managed somewhat differently. Finally, bridging policy and configurations relies on inventories that map logical entities present in the document (e.g., “Toshiba printers”) to IP prefixes. Unfortunately, the inventories may not be up to date, especially given the organic growth of enterprise networks through mergers.

Contributions. In this paper, our primary contribution is *Elastispec*, an early step towards formalizing enterprise management with *imprecise and incomplete specifications*. Elastispec enables operators to audit their configurations against natural language vendor documents while tackling the pragmatic challenges in enterprises detailed above. Elastispec does so through three components:

First, Elastispec formalizes vendor documents in a custom DSL (§3). The DSL generalizes existing commercial firewall languages [5, 18, 19, 31, 40, 48, 53, 69] by capturing the *choice* and *conditionality* inherent in the documents. For instance, the DSL captures policies of the form “the application server must talk to *at least one* of multiple database servers,” or policies that permit specified traffic *conditional* on a given application feature being enabled.



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGCOMM '26, Denver, CO, USA*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2467-1/2026/08

<https://doi.org/10.1145/3789240.3829191>

Next, Elastispec uses LLMs to (i) derive candidate DSL specifications from vendor documents; and (ii) concretize the specification for the enterprise by translating logical entity names used in the documents to actual enterprise physical IP prefixes. Our use of LLMs is motivated by the fact that manually constructing formal specifications is well known to be both prohibitive and highly error-prone [59]. Unfortunately, the obvious approach to using LLMs for the first step leads to poor accuracies owing to the need to maintain consistent conditions and entity names throughout the DSL, and infer logical conditions and optional policies. Instead, Elastispec uses a carefully crafted multi-step LLM agent based on a hierarchical representation of the vendor document. For the second step, Elastispec uses LLM agents that can correlate information across diverse data sources (vendor document, inventory file and existing configurations) to achieve much higher accuracy than relying on inventories alone.

Third, Elastispec adopts an *angelic verification* methodology (using terminology from [14, 20]) to handle *partially mapped* policies, as mapping all logical entities to IP prefixes is infeasible with available data sources. Under this framework, Elastispec identifies environmental assumptions that make the system compliant, reporting these to the operator as *conjectures*. For example, Elastispec may certify that a reachability policy pertaining to printers is satisfied under the conjecture that the printers belong to a particular subnet, leaving final validation of the conjectures to the network operator.

Results. We validate Elastispec¹ using documents of several large and widely used enterprise applications, and firewall configurations of a large distributed enterprise network. We show that Elastispec is effective in enabling operators to audit their configurations against vendor documents by producing compliance trees, enabling comparative analysis of multiple application instances, and detecting configuration errors that permit non-compliant traffic. We also show that: (i) the Elastispec DSL is able to effectively capture policies from several vendor documents; (ii) Elastispec’s LLM agent significantly increases accuracies when extracting DSL policies from 61% to 96% relative to a baseline approach; and (iii) Elastispec uncovers and analyzes twice the number of applications and 2.6X more application instances by jointly analyzing diverse data sources relative to a baseline limited by inventory files. Overall, Elastispec shows the feasibility of principled approaches to configuration management with informal and incomplete specifications.

2 Motivation

Enterprise data centers support hundreds and even thousands of applications. The applications are diverse, supported by many different vendors, and sometimes custom-built. Applications span tasks such as print management, inventory and asset management, process automation utilities, and productivity suites [17, 51, 54, 60, 70]. Balancing security needs with the correct functioning of these applications requires carefully configuring enterprise firewalls to ensure only appropriate traffic is permitted.

2.1 Enterprises Lack Precise Specifications

Configuring firewalls (or verifying their correctness) [4, 8, 9, 28–30, 42, 43, 55, 65, 76, 78] requires a specification—i.e., a list of 5-tuples

¹code available at <https://github.com/Purdue-ISL/Elastispec>

(source and destination IP prefixes, protocol, and ports) which should (or should not) be allowed to communicate. Unfortunately, obtaining such specifications is challenging (§2.1), which motivates Elastispec (§2.2).

Our discussions with operators of a large enterprise network indicate the process of deciding what traffic must be permitted by a firewall is complex and requires piecing together information spread across *diverse sources* including:

Several hundreds of vendor documents. Each enterprise application is typically associated with one or more vendor-specific natural language documents which indicate which protocols and ports must be permitted for the successful functioning of the application. The documents are in heterogeneous vendor-specific formats spanning several hundred lines. For example, the document associated with PaperCut—a large print management application—has 484 lines of English text that describe 43 different policy blocks that cumulatively make references to 128 policies, each of which corresponds to a protocol and a port (or port range).

Vendor documentation is often implicit relying on the reader’s expertise for appropriate interpretation. Consider the following policy from PaperCut’s documentation:

"PaperCut makes use of the jTDS JDBC driver for Microsoft SQL server..., the postgresql JDBC driver for PostgreSQL, Oracle JDBC for Oracle and MySQL JDBC for MySQL. The defaults are below. Oracle: 1521, Microsoft SQL Server: 1433, MySQL: 3306, PostgreSQL: 5432"

While unstated, the phrase “makes use of” implies that the PaperCut server initiates the connection to the database server, and the use of JDBC drivers implies the TCP protocol.

Undocumented local knowledge spread across multiple operator teams. Vendor documents are written more broadly, and only a subset of policies may be relevant to any specific enterprise. In the example above, the policy intent is that an administrator must choose *at least one* database backend (e.g., Oracle **OR** PostgreSQL), and not all of them need to be simultaneously selected. Thus, extracting precise reachability requirements requires not only knowledge of the above document but also local knowledge of which database is used by the local application administrator. Figure 2 presents another example where the traffic to be permitted depends on the make of the printer (Toshiba vs. Canon vs HP), the SDK version used, and whether the enterprise uses LDAP (or the more secure LDAPS) for directory access.

Our analysis of vendor documents shows that such *choice* and *conditionality* is pervasive, and the reachability requirements highly depend on the local context of the specific enterprise. Detailed knowledge of an enterprise’s local context is rarely documented, and spread across multiple operator teams. For instance, our discussions indicate that the security team which manages firewalls does not maintain records on which optional features an application administrator turns on, while the application admin that supports a print management application may not have exact knowledge of the printer make and model used by each department.

Parallel application deployments with different needs. An enterprise may have several instantiations of the same application which may potentially be configured differently. For instance, the large enterprise that we have studied is spread across *multiple*

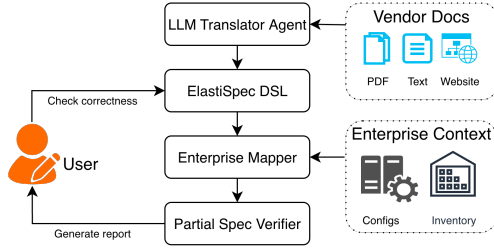


Figure 1: Elastispec overview.

geographically distributed campuses, with a large campus having multiple departments that each customize their print management application differently.

Inventories that may be incomplete or inaccurate. Vendor documents refer to logical entities (e.g. MobilityPrint Server, Lantronix etc.) while reachability requirements must be expressed at the IP level. Existing research assumes accurate mappings of logical names to physical addresses is easily available [21, 37] which may not hold in practice. Our analysis of the enterprise data center inventory indicates over 37% of the data center servers do not have a meaningful label, with a major factor being a recent merger across campuses. Further, not all organizations track assets outside the data-center in a centralized fashion, which complicates obtaining such mappings – e.g., it may be desirable to restrict access to printers, but the set of printers may be scattered across different departmental subnets.

2.2 Elastispec Rationale and Overview

Elastispec is a system that aids operators in auditing enterprise firewall configurations against policies expressed in vendor documents. Elastispec enables operators to (i) identify which policies in the vendor specification have been correctly implemented by the configurations; (ii) associate configuration rules with policies in the vendor documents; and (iii) identify configuration rules that do not have matching policies. Elastispec has the following components and workflow (summarized in Figure 1):

Formalizing vendor documents (§3). As a first step, Elastispec formalizes the vendor documents in a custom DSL (§3) that we have developed. The DSL is a generalization of commercial firewall languages in that it not only embeds reachability requirements but also captures the *choice* and *conditionality* inherent in the documents. For instance, the DSL captures policies of the form “the application server must talk to *at least* one of multiple database servers,” or policies that permit specified traffic *conditional* on a given application feature being enabled.

LLM-based formalization and mapping (§4). To convert vendor documents into the formal DSL, Elastispec uses an LLM agent that carefully accounts for the choice, and conditionality in the policies. Next, Elastispec maps abstract DSL policies to the concrete enterprise environment. Specifically, Elastispec (i) discovers distinct instances of the same application – e.g., different departments in an enterprise may run different instances of a print management application, each configured somewhat differently; and (ii) associates logical entity names in the DSL to actual physical IP prefixes. A key issue is that information needed for the mapping may not

Inbound

- 9191 “TCP – for EWB and SOAP events”
- 9192 “TCP – secure messages for EWB and SOAP events”
- 10389 “TCP (LDAP) – for SDK1 and SDK2 only”
- 10636 “TCP (LDAPS) – for SDK1 and SDK2 only”
- 162 “UDP (SNMP traps) – for SDK1 only”

Outbound

- 161 “UDP (SNMP) – for SDK1 only”
- 49629 “TCP (HTTP) – for SDK2, SDK3 and RD30 only”
- 49630 “TCP (HTTPS) – for SDK2, SDK3 and RD30 only”
- 50083 “TCP (HTTP) – for SDK3 only (for scanning)”

Figure 2: Example firewall ports for Toshiba configuration from the PaperCut vendor documentation[54].

be explicitly documented, or embedded in incomplete inventory files. Elastispec tackles this issue by jointly correlating information spread across vendor documents, server inventory files and network configuration files.

Auditing configurations with partially mapped policy (§5).

Note that despite the multimodal mapping effort from the previous component, the policy may still be partially mapped (e.g., logical entities mentioned in the DSL may not be successfully mapped to concrete IP prefixes). As a final step, Elastispec verifies the feasibility of such incomplete specifications by checking reachability under conjectured environments. For instance, Elastispec may certify that the policy that Toshiba printers must talk to Papercut servers under the conjecture that “the printers are contained in the prefix 1.2.3.4/24.” Elastispec also flags errors where required policies have not been implemented (e.g., no database can be reached), and identifies configuration rules that are not associated with any policy in the vendor document.

3 Formalizing Vendor Documents

Elastispec formalizes vendor documents into a custom DSL that we discuss in this section. The DSL avoids the inherent ambiguity of natural language by making policy logic explicit which facilitates auditing. While several commercial firewall languages exist [5, 18, 19, 31, 40, 48, 53, 69], the Elastispec DSL is motivated by the fact that vendor document policies involve significant *choice* and *conditionality* in their reachability requirements.

To illustrate, Listing 1 presents the specification for the example shown in Figure 2. We make the following points:

- The specification indicates that the policy for Toshiba printers is satisfied as long as the policy for *at least one* SDK is met. Further, the SDK3 policy makes *explicit* which inbound and outbound traffic mentioned in Figure 2 are relevant to this SDK. The policy also makes clear that it is sufficient if one of the inbound TCP ports 9191 or 9192 are reachable (R1), and one of the outbound ports 49629 or 49630 are reachable (R2).
- Line 15 in Listing 1 corresponds to 50083 TCP rule in Figure 2. Notice the use of the **optional operator (?)** at the end of this line.

```

1 def policy Toshiba as
2   SDK1 || SDK2 || SDK3 || RD30
3 def policy SDK1 as
4   ...
5 def policy SDK2 as
6   ...
7 def policy SDK3 as
8   # R1. EWB and SOAP. Unsecure vs secure
9   ((ToshibaMFD, *) -> (PapercutServer, 9191) on tcp) || ((
10    ToshibaMFD, *) -> (PapercutServer, 9192) on tcp)
11   &&
12   # R2. HTTP or HTTPS
13   (((PapercutServer, *) -> (ToshibaMFD, 49629) on tcp) || ((
14    PapercutServer, *) -> (ToshibaMFD, 49630) on tcp))
15   &&
16   # R3. HTTP Scanning (optional)
17   ((PapercutServer, *) -> (ToshibaMFD, 50083) on tcp)?
18 def policy RD30 as
19   ...

```

Listing 1: Toshiba policy in Elastispec.

This is a key aspect of Elastispec and indicates the rule is optional to configure.

Compositionality. The above example also illustrates the compositional nature of Elastispec given that Toshiba policy combines the policy for each SDK using the logical OR operator (`||`). To illustrate further, consider the top-level specification for PaperCut policy shown in Listing 2.

```

1 def policy Database as
2   Oracle || MySQL || MSSql...
3 def policy Printer as
4   Toshiba || Canon || Fuji...
5 ...
6 def policy PaperCut as
7   Database && Printer && Optional? && ...
8 assert Papercut

```

Listing 2: Top-level Policy in Elastispec.

The specification combines multiple components like `Printer`, `Database` and `Optional` using the logical AND operator (`&&`). Each of `Printer` and `Database` combine specifications for printers and databases respectively from many different vendors. Finally, notice that `Optional` is referred to with the optional operator (`?`). This indicates that the entire policy and all of its rules are optional.

Language Design. Figure 3 presents the Elastispec DSL syntax. The syntax is with respect to a fixed namespace $\mathcal{I}$, which is a set of entity names. Intuitively, an entity is a role in a network service with a unique name. Examples include `PaperCut::Server`, `Oracle::Server`, `Veeam::BackupServer`. Entities may also be `GenericEndHost` (generic host on campus), and `GenericInternetHost` (arbitrary host on the Internet).

The Elastispec is designed to describe network connectivity rules set forth by vendors in a structured and modular way. A specification may optionally define entity groups and one or more named policies, and concludes with an `assert` statement that designates a single top-level policy to be enforced. Groups are named collections of entities, while policies encode connectivity logic as boolean compositions of connectivity expressions. A salient feature of Elastispec is the `C?` construct, which means the connectivity constraint C is optional and only required in a particular context, making the

$e \in \mathcal{I}$: entity name g : group name pol : policy name
 $pr \in \{\text{tcp, udp, ip, ...}\}$: protocol $pt \in [0, 65535]$: port number
Vendor Spec $\mathcal{V} ::= \mathcal{G}^* \mathcal{P}^+ \text{ assert } pol$
Group Definition $\mathcal{G} ::= \text{def group } g \text{ as List}(e)$
Policy Definition $\mathcal{P} ::= \text{def policy } p \text{ as } C$
Connectivity $C ::= C || C \mid C \&\& C \mid$
 $(C) \mid C ? \mid pol \mid \mathcal{A}$
Atom Connectivity $\mathcal{A} ::= \mathcal{T} \rightarrow \mathcal{T} \mid \mathcal{T} \rightarrow \mathcal{T} \text{ on } pr$
Subject $\mathcal{T} ::= (e, \mathcal{R})$
Port Range $\mathcal{R} ::= \text{List}(pt) \mid pt-pt \mid *$

Figure 3: Syntax of Elastispec.

policy general and not enterprise-specific. The atomic policy unit is a directional connection between two subjects, where each subject pairs an entity with a port specification (a list, a range, or the wildcard `*`)

4 LLM-based Formalization and Mapping

We next discuss how Elastispec (i) uses LLMs to generate candidate DSL specifications from vendor documents for operator inspection; and (ii) maps the DSL to a concrete network environment.

4.1 Translating Vendor Docs to Elastispec

Why not a naive LLM approach? A natural strategy for translation is a single pass top-down approach, which feeds the entire vendor document to an LLM. In practice, this exceeds context limits and performs poorly on large documents, achieving only 61% mean flow-level accuracy (§6.4). Another approach is to split the document into chunks, generate policy fragments using an LLM, and stitch them together. Unfortunately, this bottom-up Retrieval-Augmented Generation (RAG) workflow also performed poorly owing to a lack of global context, with a mean accuracy of 55% (§6.4).

Elastispec’s approach. Given this, we developed an LLM agent which employed a mixed strategy that preserves local fidelity at leaves while enforcing global consistency during composition, achieving a mean flow-level accuracy of 96% (§6.4). The workflow is summarized in Figure 4 (detailed prompts in Appendix C)

Preprocessing and structuring. First, the raw, often messy and vendor-specific natural language documentation is converted into a clean, structured format. A structured, hierarchical JSON outline that resembles a table of contents is generated as shown in Figure 4, part (A). An LLM then extracts the text for each section using RAG.

Policy and entity extraction. Next, the agent processes each node to extract a flat list of all relevant firewall policies and network entities (e.g., “PaperCut Server,” “Oracle Database”). Each policy is assigned a unique identifier and annotated with a natural-language *active condition* describing when it applies (e.g., “when using an Oracle database”). Figure 4(B) illustrates this for the Toshiba node.

Leaf level spec generation. Next, the agent considers each leaf node and creates a leaf spec. To this end, the LLM first merges rules that have the same active condition (e.g., “SDK1”), and next combines policies using logical operators (AND, OR), creating additional branching if needed. We use prompts to alert the LLM to ensure consistency across separated policy statements (e.g., “SDK1”

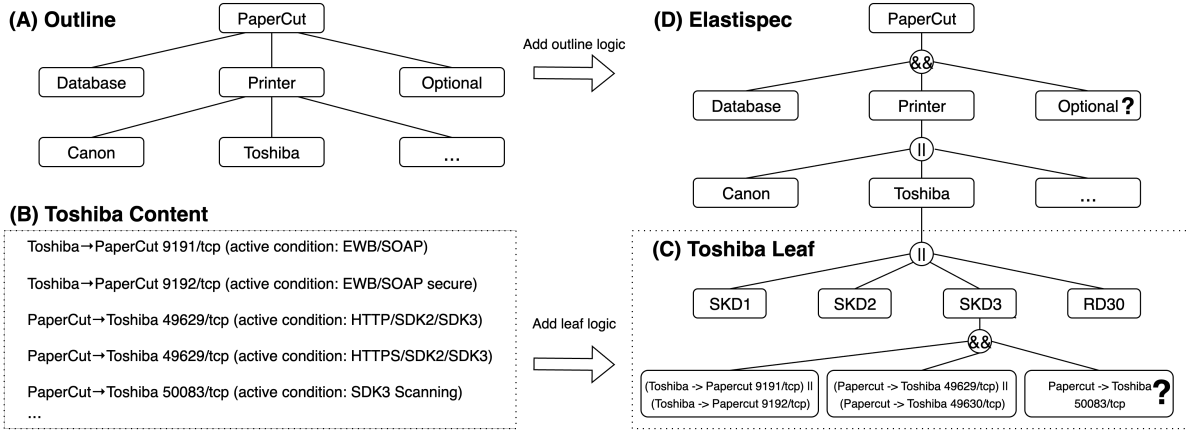


Figure 4: Illustrating multi-step process used by Elastispec’s LLM agent to generate candidate DSL.

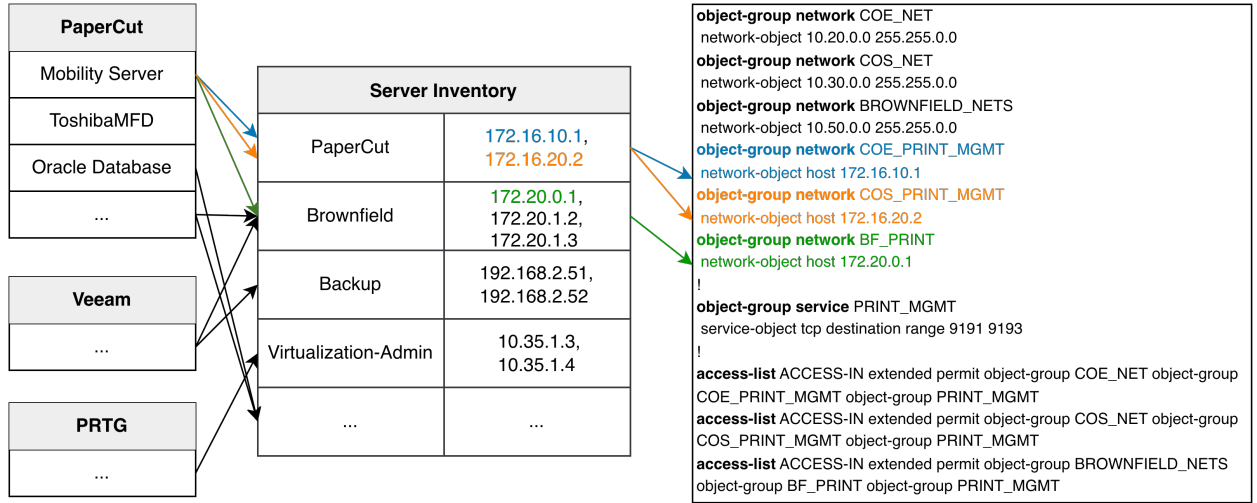


Figure 5: Elastispec maps vendor spec to network by correlating (i) logical entities extracted from vendor documents (left); (ii) inventory files that may have uninformative labels (middle); and (iii) actual configurations (right).

appears in both the inbound and outbound statement blocks in Figure 2) without which errors might result in the translation. Figure 4, part (C) illustrates the generated specification for the Toshiba node.

Hierarchical spec generation. Next, the agent assembles a complete specification by organizing all leaf specifications into a coherent hierarchy. It uses the document’s outline generated in the first step to determine the overall structure and inferring the logical relationships connecting children at each parent node. As shown in Figure 4, part (D), a coherent hierarchy is generated for the PaperCut application. This final step also analyzes the complete specification in the context of the full documentation to determine the optional status for every single spec in the hierarchy.

4.2 Mapping Vendor Documents to Network

In this section, we discuss how Elastispec maps the formalized vendor document to the concrete enterprise network. This requires Elastispec to map relevant logical entities in the DSL to their IPs.

Our analysis of vendor documents indicates entities belong to a few categories: (i) application server related (e.g., PaperCut, MobilityServer); (ii) other data center servers (e.g., OracleDB, LDAPServer); and (iii) client entities (e.g., PaperCut Client, EmbeddedMFD, ToshibaMFD etc.). Client entities are not easily resolvable in general since vendor labels of client entities (e.g., EmbeddedMFD) may not correlate with labels used in IP Management databases (e.g., the embedded devices may be spread across several subnets, and an explicit database of these devices may not be maintained). We next discuss how Elastispec resolves server entities by leveraging inventories, and defer how it handles unresolved client entities to §5.

The main issues in mapping server entities are (i) handling multiple deployments of applications; and (ii) dealing with uninformative inventory entries. For example, Figure 5 shows an inventory where generic labels (e.g., “Brownfield” and “Virtualization-admin”) do not convey meaningful application information. We discuss how Elastispec handles each of these in turn.

Inferring multiple instances. Elastispec categorizes servers sharing the same inventory label as belonging to the same instance if they have identical reachability requirements for all traffic [12, 41], and assigns them to different instances otherwise. For example, Elastispec separates out the servers sharing each of the labels “PaperCut” and “Brownfield” into different instances (shown by different colors).

Handling uninformative inventory entries. The key idea that Elastispec leverages is that the configuration files embed port numbers associated with the servers in an instance, which in turn forms a *signature* that may help identify an application. For example, Figure 5 shows that although the server 172.20.0.1 (green) has an uninformative “Brownfield” label, it is associated with the ports 9191 to 9193, which typically indicate a PaperCut server. Based on this insight, Elastispec uses an LLM agent to infer the application(s) corresponding to each instance based on the associated ports, and creates an *enriched inventory label*.

The agent filters out common <prot,port>pairs (e.g., TCP/80, TCP/443) and queries an LLM to identify a minimal set of applications that explain most observed ports. We found that a direct LLM query only resulted in an accuracy of under 50%. To achieve higher accuracy, our LLM agent (i) creates a merged candidate list by querying two LLMs; (ii) builds an application–port matrix by querying if each candidate application in the earlier step is associated with a particular <prot,port>pair; (iii) scores candidate applications based on port coverage weighted by port uniqueness; and (iv) applies a greedy weighted set-cover algorithm that covers most ports with the fewest applications. We found this procedure improves accuracy to 93%. We provide more details and accuracy results in Appendix A.

5 Auditing Enterprise Firewalls

Given an Elastispec specification and mapping, the next step is to audit the enterprise’s firewall configurations against the inferred policies. If a policy is given in terms of 5-tuples (source/destination IPs, source/destination ports, and protocol), verification is well studied. [2, 29, 42, 43, 78]. However, two obstacles must be addressed. First, even with the multimodal mapping approach described in §4.2, the document-to-network mapping usually remains incomplete (e.g., with unresolved client entities), preventing the direct application of these existing verification tools. Second, a policy is by nature hierarchical, consisting of multiple layers of policies with conditionals and options (§3). Accordingly, the auditing output should also be structured hierarchically, rather than reduced to a simple yes/no determination of whether the policy matches.

The Elastispec auditor tackles these issues by leveraging an angelic verification framework to find valid environmental assumptions (conjectures) that satisfy the policy. We begin by formalizing the verification problem and then present our auditing approach.

5.1 Framing the Angelic Verification Problem

Now let us formally define what Elastispec checks when a configured firewall is given. As mentioned earlier, checking whether a specific 5-tuple is viable in a configured network is a standard network verification problem. Therefore, we assume the existence of an oracle (e.g., Batfish) that is capable of answering queries regarding 5-tuple connectivity. When the output of the enterprise mapper component still involves unresolved entities, the auditing component of Elastispec Elastispec can be framed as an instance of *angelic verification* [14, 20]. Intuitively, the verifier treats the missing mapping information as angelic non-determinism, searching for a conjectured environment where the unresolved entities can be mapped to IP addresses such that the resulting 5-tuples are satisfied by the configured network. We formalize below.

DEFINITION 1 (CONNECTIVITY ORACLE). A *connectivity oracle* is a set of 5-tuples $O \subseteq \text{IPs} \times \text{IPs} \times \text{Ports} \times \text{Ports} \times \text{Protocols}$. O is said to be *consistent with a configured network* if for each 5-tuple $q = (v_1, v_2, pt_1, pt_2, pr) \in C$, $q \in O$ if and only if the traffic characterized by q is permitted in the network.

DEFINITION 2 (ENTITY MAPPING). An *entity mapping of an entity namespace $\mathcal{I}$* is a (total or partial) function $M : \mathcal{I} \rightarrow 2^{\text{IPs}}$ from entity names to sets of IPs.

An entity mapping is said to be *total* (resp. *partial*) if the function M is total (resp. M is partial). An entity mapping M_1 is an *extension* of another mapping M_2 , denoted as $M_2 \sqsubseteq M_1$, if the domain of M_1 is a superset of the domain of M_2 , and $M_1(i) = M_2(i)$ for every i in the domain of M_2 .

Now we are ready to define the Elastispec verification problem. Intuitively, an Elastispec verifier should return VALID if a network configuration (characterized by O) is guaranteed to meet the specification (φ with current mapping M), or return UNSAT if the configuration is guaranteed to violate the specification, or return CONJECTURED-SAT if the current mapping is not complete and the result depends on how we interpret those unresolved entities. In the last case, the verifier should present a conjectured mapping with which the specification can be satisfied.

DEFINITION 3 (THE VERIFICATION PROBLEM). An *Elastispec verification problem* is represented as a tuple $(\mathcal{I}, \varphi, M, O)$ where $\mathcal{I}$ is an entity namespace, φ is an Elastispec specification w.r.t. $\mathcal{I}$, M is an entity mapping of $\mathcal{I}$, and O is a connectivity oracle. The verification problem is to produce the following outputs by making queries to the oracle O :

- if for every total mapping M^* such that $M \sqsubseteq M^*$, $M^* \models_O \varphi$, then return VALID ;
- if for every total mapping M^* such that $M \sqsubseteq M^*$, $M^* \not\models_O \varphi$, return UNSAT ;
- otherwise, return CONJECTURED-SAT along with a witness mapping M^* such that $M \sqsubseteq M^*$ and $M^* \models_O \varphi$.

where the judgment $M^* \models_O \varphi$ indicates that a total mapping M^* satisfies specification φ as per the connectivity oracle O . We formally define the $\models_O$ judgment below.

The definition essentially gives the formal semantics of Elastispec modulo 5-tuple connectivity. Let us start from the atomic connectivity. Intuitively, a total deployment satisfies an atomic connectivity if every corresponding 5-tuple is permitted as per the oracle:

$M \models_O (e_1, \mathcal{R}_1) \rightarrow (e_2, \mathcal{R}_2) \text{ on } pr$ iff. for any 5-tuple $c = (v_1, v_2, pt_1, pt_2, pr)$ that satisfies the following conditions, $c \in O$: • $v_1 \in M(e_1)$ and $v_2 \in M(e_2)$; • $pt_i \in \mathcal{R}_i$, if $\mathcal{R}_i$ is a list or range of ports, for $i \in \{1, 2\}$.
--

Operators $\parallel$ and $\&\&$ are interpreted as logical disjunction and conjunction as typical. Note that a connectivity $C?$ is optional and can be completely ignored—i.e., $M \models_O C?$ always holds. The semantics of a policy p is determined by its body C , and the semantics of a specification $\mathcal{V}$ is determined by its top-level policy assertion p .

5.2 Auditing Methodology

We now discuss how Elastispec solves the angelic verification problem and audits an Elastispec specification against a firewall. Serving as a verifier (cf. Definition 3), the auditing component takes as input the formal specification ϕ from the translator (§4.1), the entity mapping M per instance from the mapper (§4.2), and employs Batfish [29] as the connectivity oracle. For each instance, the auditor produces not only the verification result for the whole specification (VALID, UNSAT, or CONJECTURED-SAT), but also results for each constituent policy or subpolicy, forming a hierarchical structure we call *compliance tree*. In addition, the auditor checks the reverse direction and detects traffic allowed but not covered by the specification.

For concreteness, we discuss Elastispec’s approach by example, in the context of the Toshiba printer policy shown in Listing 1, and explain how Elastispec determines whether the policy is potentially satisfiable. We leave the algorithmic description in Appendix B.

Conjecturing for incomplete mapping. Consider a specific connectivity (ToshibaMFD, *)->(PaperCutServer, 9191) on tcp. If both entities were successfully mapped, the connectivity can be directly verified via a series of queries to Batfish [29] (or other existing verification tools). However, consider a scenario where for a particular PaperCut instance, PaperCutServer has been mapped to {172.16.10.1}, but entity ToshibaMFD has not been mapped. A key task for Elastispec is to *conjecture* a mapping for ToshibaMFD so that the connectivity can be verified. Elastispec tackles this by determining the set of IPs that can reach the specific PaperCut servers ({172.16.10.1}), and indicating that the policy is satisfied under the conjecture that ToshibaMFD is mapped to these IPs. To determine this set, Elastispec computes, for this leaf, a set of all possible IPs that satisfy the reachability requirement. Concretely, after identifying the relevant ACL and service constraint, Elastispec leverages Batfish’s internal symbolic analysis to derive a BDD that characterizes exactly the source IPs from which traffic can reach the mapped PaperCut server on the given protocol and port. We use this BDD as the conjecture for the unmapped role. If the resulting BDD is empty, then no conjectured mapping can satisfy the leaf and the policy is UNSAT. Otherwise, the leaf is CONJECTURED-SAT, together with the symbolic IP set represented by the BDD. Because the BDD ranges only over the unmapped endpoint IP space for this fixed

leaf, it compactly represents potentially many satisfying subnets without enumerating them individually.

Verifying composite policies. We next consider whether the composite policy SDK3 shown in Listing 1 is satisfied, continuing with the general case that ToshibaMFD is not mapped. As ToshibaMFD is involved in multiple atomic connectivity statements, Elastispec will verify them separately. In most cases, the verification results can be assembled algebraically following the natural semantics of $\parallel$ and $\&\&$. Let B_1 , B_2 , and B_3 denote the conjecture BDDs for R1, R2, and R3, respectively. Here, B_1 is the union of the two leaf-level conjectures arising from the two branches in R1, and similarly B_2 is the union of the two leaf-level conjectures arising from the two branches in R2. Since R3 is optional for SDK3, it does not constrain satisfiability of the whole SDK3 policy. Thus, the most noteworthy scenario is the $\&\&$ combination of the required subpolicies R1 and R2 when they are both CONJECTURED-SAT. In that case, Elastispec checks whether the conjunction remains satisfiable under a single mapping of ToshibaMFD, i.e., whether $B_1 \cap B_2 \neq \emptyset$. If so, SDK3 is also CONJECTURED-SAT. If in addition $B_1 \cap B_2 \cap B_3 \neq \emptyset$, then the optional rule R3 is also reported as CONJECTURED-SAT; otherwise, it may simply be ignored without affecting satisfiability of SDK3.

Constructing compliance trees. Using the composite-policy verification approach described above, Elastispec constructs a compliance tree for each application instance, reflecting the hierarchical structure of vendor policies. The tree summarizes which policies are satisfied, which hold only under specific conjectures, and which are violated. Such trees, as shown in Figure 7, provide enterprises with an explicit, machine-generated record of the key configuration decisions—e.g., which printers or SDKs are supported and which databases an application server may access—information that operators indicated is valuable but burdensome to document manually.

Detecting out-of-spec flows. Besides detecting whether vendor policy has been implemented in configurations, Elastispec identifies *out-of-spec* flows permitted by the configuration but not justified by the vendor documentation. To do so, Elastispec analyzes the ACLs on the interface(s) connected to the application servers and compares all rules involving these servers against the set of flows matched by the vendor specification. Any ACL rule that permits traffic to or from the application servers but not specified in vendor document is reported as out of spec. Note that in making this assessment, Elastispec looks across vendor documents of all applications provided to it to account for *cross-application communication patterns*. For example, when analyzing a PaperCut application, traffic from other applications such as Veeam (a backup service) and PRTG (monitoring) is considered as meeting specifications as long as it conforms to the documents of those applications.

6 Evaluation

We evaluate Elastispec’s effectiveness in enabling operators to audit their configurations against vendor policy documents with specifications that are imprecise and incomplete.

Enterprise dataset and methodology. We evaluate Elastispec using data obtained from a large scale geographically distributed enterprise with a main campus comprising multiple departments, and multiple satellite campuses. The enterprise is in the process

Table 1: Expressiveness of Elastispec. Key features of vendor documents, and Elastispec specifications.

Application	Document features			Elastispec specifications					
	Description	Policy Blocks	Doc Words	Flows			Nodes	LoC	Depth
				Req	Opt	Total			
PaperCut [54]	Print management	42	13,343	9	117	126	53	108	4
HP WebJetadmin [35]	Web printer admin	25	1,337	4	27	31	26	54	3
Veeam [70]	Backup and recovery	28	4,185	0	55	55	36	74	4
Sassafras [60]	License management	6	1,685	18	10	28	18	38	4
LSoft [44]	Mailing automation	14	3,494	8	10	18	14	30	3
CCure [39]	Access control	269	6,348	1	288	289	248	498	3
EZProxy [51]	Library proxy	3	147	2	2	4	5	12	3
ArcGIS [27]	GIS platform	6	301	0	8	8	8	18	3
PRTG [52]	Network monitoring	60	2,222	9	76	85	63	128	3
UC4 [17]	Job scheduling	2	868	0	11	11	7	16	3

of consolidating its operations, moving away from decentralized administration across campuses and departments. The enterprise recently shifted to centralized administration of firewalls with the centralized IT security inheriting the legacy rules from other campuses. Our evaluations focus on the following themes:

DSL expressiveness (§6.1) and accuracy of LLM-based formalization (§6.4). We evaluate whether the Elastispec DSL can capture policies of popular enterprise applications, and the accuracy attained when formalizing with LLMs. Our evaluations are based on publicly available vendor documents of many widely used enterprise applications (e.g., [54, 70]) currently deployed in the enterprise. The applications were chosen based on their scale, the range of typical enterprise use cases covered, and rich communication patterns, and complex dependencies.

Mapping DSL with imperfect inventories (§6.2). We used inventory files provided by IT Security that mapped data center servers to their purpose. 37% of the servers do not have a meaningful label with the recent merger being a key reason. The enterprise often had many deployments of the same application across different campuses and departments, each configured somewhat differently. We extracted distinct application instances, and created an *enriched inventory* to map servers to their role based on §4.2 (results in §6.2).

Auditing firewall configurations with informal and incomplete specifications (§6.3). We evaluated Elastispec’s ability to audit configurations of 70 firewalls across all campuses against the vendor docs above working closely with operators over an 18 month period to obtain their feedback on Elastispec’s auditing results.

6.1 Expressiveness of Elastispec

Table 1 summarizes the enterprise applications analyzed, characteristics of their vendor documents, and key features of the corresponding Elastispec specifications. The specifications were generated as candidate DSL specifications using our approach discussed in §4.1 and then manually reviewed and corrected (serving as ground truth for the rest of the evaluations) as we elaborate in §6.4. Table 1 shows the number of words in the vendor document, the number of distinct policy blocks, and the number of 5-tuple flow rules with a breakdown of flows that are required and optional. We also summarize the Elastispec specification, including the number of nodes, the LoC, and the tree depth in the hierarchical representation.

The table shows that Elastispec is expressive and captures policies in a wide range of application documents, ranging from simple ones like EZProxy with 4 total flows to more complex ones like CCure with 289. The table also highlights the extensive prevalence of choice and optionality, underscoring the importance of Elastispec’s emphasis on these aspects. For CCure and Veeam the specification consists almost entirely of optional policies. The table also shows the utility of Elastispec’s compositional nature to manage policies, with a tree depth of 3 or 4 for all applications. Finally, Elastispec specifications are concise—on average, a single line corresponds to over 34 words of vendor documentation.

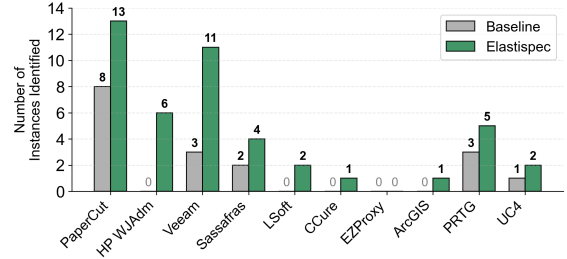


Figure 6: Elastispec uncovers more instances of many applications relative to a baseline that relies on server inventories that may have uninformative labels.

6.2 Mapping DSL to network

We next evaluate the effectiveness of Elastispec in uncovering instances corresponding to the applications above. Recall that Elastispec (i) identifies distinct instances associated with raw inventory labels (which may be overly generic and not associated with any application); (ii) creates an enriched inventory that annotates servers in each instance with a likely application inferred using a custom LLM agent described in §4.2; and (iii) uses an LLM to map entities to enriched inventory labels. We compare Elastispec with a baseline that uses an LLM to map each entity to raw inventory labels.

Figure 6 shows the number of instances of each application uncovered by Elastispec and the baseline. First, the baseline is unable to identify even one instance for five of the applications because the servers were associated with generic labels. Second, Elastispec is able to identify at least one instance for 9 applications, and identifies 45 instances in all, compared to the baseline that

```

▼ PaperCutFirewallSpec (CONJECTURE)
  ▼ AND [CONJECTURE]
    ▼ MainApplicationServerPortsSpec (CONJECTURE)
      ▼ AND [CONJECTURE]
        ((PaperCut_Internal_Clients, *) → (PaperCut_Application_Server, 9191) on tcp) [CONJECTURE]
        ((PaperCut_Internal_Clients, *) → (PaperCut_Application_Server, 9192) on tcp) [CONJECTURE]
        ((Embedded_MFDs, *) → (PaperCut_Application_Server, 9193) on tcp) [CONJECTURE]
        ((PaperCut_Application_Server, *) → (Global_Entitlements_Service, 443) on tcp) ?
      ► CardReadersSpec ?
      ► ExternalDatabaseSpec
    ...
  ▼ MobilityPrintFirewallRulesSpec ? (CONJECTURE)
    ▼ AND [CONJECTURE]
      ► MobilityPrintFirewallRules_Auto_updateSpec
      ▼ (group)
        ▼ OR [CONJECTURE]
          ► MobilityPrintFirewallRules_mDNSPrinterDiscoverySpec
          ▼ MobilityPrintFirewallRules_KnownHostPrinterDiscoverySpec (CONJECTURE)
            ▼ AND [CONJECTURE]
              ((Mobility_Print_Server, *) → (PaperCut_Client_Download_Services, 443) on tcp)
              ((PaperCut_Clients, *) → (Mobility_Print_Server, 9163) on tcp) [CONJECTURE]
              ((PaperCut_Clients, *) → (Mobility_Print_Server, 9164) on tcp) [CONJECTURE]
            ► MobilityPrintFirewallRules_DNSPrinterDiscoverySpec
            ► MobilityPrintFirewallRules_CloudPrintPrinterDiscoverySpec

```

(a) Instance from the main campus A.

```

▼ PaperCutFirewallSpec (UNSAT)
  ▼ AND
    ▼ MainApplicationServerPortsSpec
      ▼ AND
        ((PaperCut_Internal_Clients, *) → (PaperCut_Application_Server, 9191) on tcp) [CONJECTURE]
        ((PaperCut_Internal_Clients, *) → (PaperCut_Application_Server, 9192) on tcp) [CONJECTURE]
        ((Embedded_MFDs, *) → (PaperCut_Application_Server, 9193) on tcp)
        ((PaperCut_Application_Server, *) → (Global_Entitlements_Service, 443) on tcp) ?
      ► CardReadersSpec ?
      ► ExternalDatabaseSpec
    ...
  ▼ MobilityPrintFirewallRulesSpec ?
    ▼ AND
      ► MobilityPrintFirewallRules_Auto_updateSpec
      ▼ (group)
        ▼ OR
          ► MobilityPrintFirewallRules_mDNSPrinterDiscoverySpec
          ▼ MobilityPrintFirewallRules_KnownHostPrinterDiscoverySpec
            ▼ AND
              ((Mobility_Print_Server, *) → (PaperCut_Client_Download_Services, 443) on tcp)
              ((PaperCut_Clients, *) → (Mobility_Print_Server, 9163) on tcp)
              ((PaperCut_Clients, *) → (Mobility_Print_Server, 9164) on tcp)
            ► MobilityPrintFirewallRules_DNSPrinterDiscoverySpec
            ► MobilityPrintFirewallRules_CloudPrintPrinterDiscoverySpec

```

(b) Instance from a satellite campus C.

Figure 7: Compliance trees for two PaperCut instances generated by Elastispec.

only identifies 17 instances. Third, Elastispec is able to uncover co-hosted services – e.g., we identified 3 distinct instances concurrently hosting both PaperCut and HP Web Jetadmin. Finally, Elastispec is unable to identify any instance for EZproxy because its vendor documentation is poor and does not include many ports that the application requires in practice. We presented our results to the operators who manually verified them.

6.3 Auditing with Elastispec

We next investigate Elastispec’s effectiveness in auditing enterprise configurations against policy documents: (i) generating compliance trees indicating which policies are implemented; (ii) identifying necessary policies that have not been implemented; and (iii) identifying traffic permitted outside the vendor policy documents.

Creating compliance trees. We generate compliance trees with Elastispec at the granularity of individual instances. Figure 7a presents the compliance tree for one instance of the Papercut application. The tree follows the hierarchical Elastispec specification, with each node in the hierarchy combining the results of child nodes using logical conditions. Red indicates the UNSAT results; green indicates VALID or CONJECTURED-SAT (indicated by a tag next to the node). Notice that CardReadersSpec is UNSAT because no traffic can reach the server on any of the associated ports. The ExternalDatabaseSpec is VALID because Elastispec resolves both communication end points, and is able to verify that the Papercut server can reach a database.

The figure also shows that rules related to PaperCut clients are CONJECTURED-SAT. This is because although a mapping for client entities is unavailable (§4.2), Elastispec was able to find an assignment to PaperCut clients that met the specification. Specifically, Elastispec indicated that the policies could be met by any host on campus (and none outside). Our discussions with operators indicated this was reasonable since print services were available to any campus host. Finally, Elastispec was able to certify that PAPER-CUTFIREWALLSPEC is CONJECTURED-SAT because it could find a valid assignment for the set of unresolved entities that could satisfy the entire spec (§5.1 Definition 3).

Table 2: Analyzing configuration drift across application instances with Elastispec

Location	# of Inst.	App Server	Print Deploy	Mobility Print	MFD Integ.	# Unsat Rules
Campuses A1-A8	8	✓	✓	✓	✓	15
Campus A9	1	✓	✓	✓	✓	18
Campus B	1	✓	✗	✗	✓	30
Campuses C1-C2	2	✓	✓	✗	✓	29
Campus C3	1	✓	✓	✗	✗	36
Campus C4	1	✗	✗	✗	✗	72

More generally, Elastispec leaves it to operators to inspect the assignment to decide whether it is acceptable. Minimizing operator burden when evaluating conjectures is an interesting area for future research. A potential direction is to place guardrails and surface overly broad conjectures – e.g., in this enterprise, operators were particularly concerned about rules that permitted traffic from external Internet hosts outside the campus, and they indicated that highlighting conjectures that involved such cases would be of particular value. Another use case is to facilitate troubleshooting – e.g., an operator may analyze whether a conjecture is too narrow when a user indicates they are unable to access a service.

Comparing compliance trees to identify departures from common practice. Elastispec can help operators compare compliance graphs of multiple instances of the same application, a key first step toward *consolidating networks* under a common policy.

Figures 7a and 7b visually show the differences between snippets of the compliance tree of an instance from the main campus and an instance from a satellite campus. Notably the satellite campus does not support multi-function devices (EMBEDDEDMFD) or the MOBILITYPRINT service (which allows users to print from their personal devices). The operators hypothesized that the legacy instance that they had inherited potentially did not support EMBEDDEDMFD devices, a departure from the norm. They also indicated that future deployment of multi-function devices could have led to hard to diagnose trouble tickets – Elastispec averts this by proactively alerting operators to the deviant configuration.

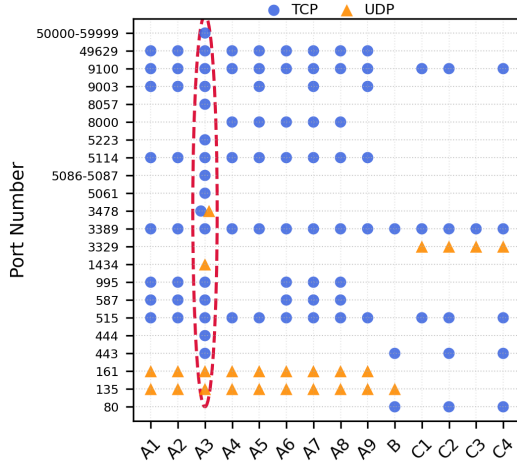


Figure 8: Out-of-spec protocol-port pairs across instances of PaperCut. The anomalous behavior of instance A3 was tracked to a configuration error.

Tracking configuration drift across instances. Given the large number of application instances, each managed a bit differently, Elastispec can analyze *configuration drift* across instances. Table 2 groups PaperCut instances by campus and summarizes key distinctions. Instances within a group are configured identically with respect to the vendor policy document, though they may differ in out-of-spec policies or in the set of client addresses accessing the servers. Each row reports the number of instances, whether traffic related to a key set of optional features is enabled, and how many leaf-level reachability policies from the PaperCut specification are unsatisfied. The operators indicated that such insights are useful, as requirements of different departments continually evolve.

Detecting configuration error through deviant out-of-spec traffic. We next analyze traffic permitted by the configurations but not matching the specifications, which may represent potential security vulnerabilities *excluding cross-application communication patterns*. (§5.2). Figure 8 plots the out-of-spec <prot,port>pairs for each PaperCut application instance. Interestingly, Instance A3 saw a much higher number of out-of-spec pairs than others. Further investigation showed that the PaperCut servers in this instance were accessible via a wide range of ports unrelated to their functionality. Of more concern, the servers had public IP addresses and allowed traffic from any Internet host on these ports, a serious departure from best practice. After analysis, the operators confirmed that these rules were intended for decommissioned public-facing servers that previously had that IP address. The servers had since been uninstalled, but the firewall policy was not updated when the IP addresses of those servers were reassigned to PaperCut.

Other configuration errors. Figure 8 also enables operator to quickly separate intended policy from unintended errors. Examples of intended policies (that are typically undocumented) include permitting the Remote Desktop Protocol (TCP/3389) for remote management, and use of ports such as TCP/9100 and TCP/515 for print servers. Table 3 details unintended misconfigurations identified by operators. One common error is **flipped direction**: e.g.,

permitting inbound access for Toshiba printer (e.g., TCP/49629) when the documentation requires outbound access. Another common error relates to **legacy configuration**, where firewall rules from older application versions or deprecated services persist. We also observed errors such as allowing both TCP and UDP on port 135 for PaperCut’s RPC service when only TCP is required, or the unneeded SNMP-Trap port TCP/162 for Veeam.

6.4 Accuracy of LLM-based DSL generation

We next evaluate the effectiveness of Elastispec’s multi-stage agent-based LLM approach (§4) in translating vendor documents to our DSL. We compare Elastispec with two baselines: (i) *Top-down*, which feeds the entire document to an LLM with the DSL policy generated by the LLM in a single pass; and (ii) *Bottom-up*, which splits the document into fixed-size chunks, with policies synthesized from these chunks using RAG.

Evaluating the accuracy of LLM-generated formal specifications is a challenge since true user intent cannot be automatically extracted [45]. Since objective ground truth is unavailable, two authors independently compared and corrected each candidate specification against the vendor document. The differences were reconciled, and when needed (e.g., owing to document ambiguity), a domain expert in charge of enterprise firewall policy was consulted. The final corrected specification was used as the ground truth.

We categorize errors with generated DSLs as follows:

- **Added flow.** The most potentially serious error from a security perspective arises when the generated spec includes flows not present in the document. Such errors are hard to detect with no obvious trigger that would alert operators to their presence.
- **Missed flow.** When the DSL does not capture a flow, the primary risk is a connectivity issue. Connectivity issues impact operations, but are less critical from a security perspective, and easier to detect (e.g., a trouble ticket indicating an inaccessible service).
- **Reqd $\rightarrow$ Opt., AND $\rightarrow$ OR.** The generated specification may be weaker than desired by marking a required flow as optional, or by replacing an AND with an OR risking connectivity issues.
- **Opt $\rightarrow$ Reqd, OR $\rightarrow$ AND.** A generated specification may erroneously strengthen logical or optionality requirements. The main risk is false positives, less serious than those above.

Breakdown of errors with Elastispec. Table 4 breaks down the errors when generating DSL with Elastispec using Gemini-2.5-Pro for the ten applications. We make the following points.

First, five of the applications do not see any error, and most applications see relatively few errors. The vast majority of errors pertain to the large CCure application (table-heavy 28 page PDF). Unlike more recent models, Gemini 2.5 Pro lacked a native document understanding interface requiring us to convert PDF (including tables) to text which impacted performance (we discuss further below).

Second, *added flows* – the most critical specification error – are less common with only 6 errors across all applications.

Third, roughly half of the remaining errors impacted connectivity, with 73% of these corresponding to missed flows. Finally, a large fraction of errors pertain to stronger logical and optionality conditions, less critical since the primary risk is false positives.

Benefits of recent LLM models. We next consider the potential of more recent LLM models. We focus on added flows (given the

Table 3: Examples of out-of-spec configuration errors detected with Elastispec

Protocol	Port	Direction	Application	Description
TCP	5114	inbound	PaperCut	Legacy ACL rule from an older version of PaperCut.
TCP	587			Legacy ACL rules. Historically, PaperCut servers acted as a mail server to receive emails, but now they watch a central mailbox.
TCP	995			Legacy configuration or addition printing service hosting on the server.
TCP	631			
UDP				
TCP	49629			Should be an outbound rule to the Toshiba printers.
TCP	515			Legacy ACL rule. Line Printer Daemon used to submit jobs to printers.
TCP	9100			Should be an outbound rule to the printers.
UDP	135			Both TCP and UDP 135 are added but only TCP is needed for RPC.
UDP	161			Should be outbound when PaperCut reaches out to the printers.
TCP	80	internal		Legacy ACL rules. These rules have been used during a migration so that some old servers could talk to the new servers, but are no longer needed.
TCP	443			
UDP	3329	inbound	HP WJAdm	App configuration error.
TCP	6183-6193	bidirectional	Veeam	Legacy ACL rule from an older version of Veeam.
TCP	162			TCP 162 is SNMP-Trap and not needed for Veeam.

Table 4: Breakdown of errors when generating DSL with Elastispec using Gemini 2.5 Pro. The errors are broken down into 6 categories (Row 2) while Row 1 indicates the worst-case impact of each category. Added flows have the most serious worst-case impact potential (security issues) but are relatively fewer.

App.	Security	Connectivity		False positive			Total
	Added flow	Missed flow	Reqd → Opt.	AND → OR	Opt. → Reqd	OR → AND	
LSoft	0	0	0	0	0	0	0
EZProxy	0	0	0	0	0	0	0
ArcGIS	0	0	0	0	0	0	0
UC4	0	0	0	0	0	0	0
HP	0	0	0	1	0	0	1
Veeam	1	0	0	0	0	0	1
PRTG	1	8	0	0	0	1	10
PaperCut	1	1	3	1	6	0	12
Sassafras	0	3	7	1	0	3	14
CCure	3	33	1	3	43	1	84
Total	6	45	11	6	49	5	122

Table 5: Comparing errors for the two most important categories (added and missed flows) when generating Elastispec DSL across two Gemini versions. With 3.1-Pro, there are no added flows, and substantially fewer missed flows.

App.	2.5-Pro A / M	3.1-Pro A / M	App.	2.5-Pro A / M	3.1-Pro A / M
PaperCut	1 / 1	0 / 0	CCure	3 / 33	0 / 0
HP	0 / 0	0 / 0	EZProxy	0 / 0	0 / 0
Veeam	1 / 0	0 / 0	ArcGIS	0 / 0	0 / 0
Sassafras	0 / 3	0 / 3	PRTG	1 / 8	0 / 1
LSoft	0 / 0	0 / 0	UC4	0 / 0	0 / 0

potential security impact), and missed flows (predominant cause for connectivity-impacting errors with 2.5-Pro). Table 5 shows that

Gemini-3.1-Pro substantially reduces these two most important error categories. Added flows are eliminated entirely, dropping from 6 with Gemini-2.5-Pro to 0 with Gemini-3.1-Pro. Missed flows are also reduced from 45 to 4, with 8 applications seeing no missed flows. The improvement is particularly strong for CCure (36 added/missed flows with 2.5-Pro vs 0 with 3.1-Pro) and PRTG (9 vs 1). Besides the improvements of the newer model itself, these applications involved table-heavy PDFs, and benefited by 3.1-Pro’s support for direct processing of PDFs.

Comparison with baselines. We next compare Elastispec with the baseline approaches (Top-down and Bottom-up). To summarize added and missed flows, we report the Jaccard similarity between the sets of flows from the generated (S_{gen}) and ground-truth (S_{gt}) specs: $J = \frac{|S_{gen} \cap S_{gt}|}{|S_{gen} \cup S_{gt}|}$. Figure 9 compares Jaccard similarity distributions across applications achieved by Elastispec and the two baselines for both the 2.5-Pro and 3.1-Pro models. We make several observations. First, for the 2.5-Pro model, Elastispec achieves a mean Jaccard similarity of 96%, while the corresponding values are only 61% for Top-down and 55% for Bottom-up. Second, while the baseline approaches improve with 3.1-Pro (with Top-down and Bottom-up achieving average accuracies of 64.78% and 73.46% respectively), Elastispec maintains a clear lead with an average accuracy of 98.67% across applications. This indicates its multi-stage workflow offers benefit even with stronger models such as 3.1-Pro.

The top-down method processes the entire document at once and often misses relevant flows in large specifications (e.g., it achieves under 10% similarity for CCure with 2.5-Pro). The bottom-up method lacks global context, leading to inconsistent assumptions and erroneous flows. For example, for the Toshiba policy (Figure 2), the chunk does not contain information about both communicating endpoints. Further, the lack of context complicates the LLM’s ability to combine flows from different chunks.

In contrast, Elastispec’s multi-stage approach handles these issues by synthesizing the outline and extracting only relevant context for each node, and then allowing the appropriate context to be utilized in a RAG pipeline in later steps. Interestingly, although

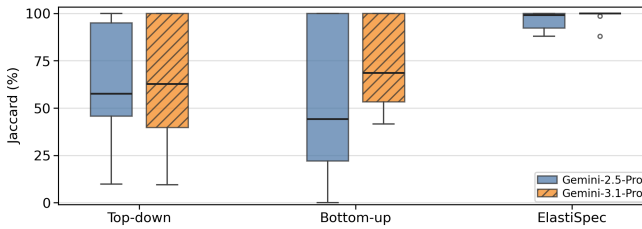


Figure 9: Distribution of per-application Jaccard similarity across the 10 applications for the two baselines and Elastispec under Gemini-2.5-Pro and Gemini-3.1-Pro.

Top-Down improves on average, some applications see worse performance. This is because the approach shows substantial variance across runs. While multiple runs could help, this would require manual correction of every run which is onerous in practice.

Correcting errors. In practice, we expect operators to inspect and correct LLM-generated specifications. Our experience suggests the time taken for correction depended on the size of the vendor document, and the number of errors in the DSL generation. Correcting Elastispec generated DSL took about 1-1.5 hours for CCure (the largest document involving a 28 page PDF), and under half an hour for all other documents, taking only a few minutes in many cases. The alternative to Elastispec’s LLM workflow is for operators to write the candidate DSL manually from scratch. For PaperCut, the manual process took over 5 hours for authors familiar with the document and the DSL (correction took ~ 30 minutes), and would be prohibitive for CCure. While we defer a systematic study of the savings to future work, our experience is consistent with others who have indicated writing specifications manually is prohibitive and error-prone [59].

7 Related work

DSLs for networking. Several DSLs have emerged for network management in recent years. An early DSL [33] allows flow-level reachability policies to be expressed while a recent DSL [68] focuses on ACL updates. Many DSLs (notably [50]) focus on policies that can be compiled to Openflow switches – e.g., Pyretic [50] allows developers to compose modules for different network functions (e.g., firewalls, load balancers). Beyond security, P4 [15] allows programming of packet forwarding planes, and DSLs have been developed for routing policies (e.g., [10]). In contrast, Elastispec’s DSL captures vendor documents focusing on security policy. A highlight is it handles the rich choice and optionality in these documents rather than precise flow-level reachability requirements (which may not be known).

Network verification. Much progress has been made in recent years towards formal verification of network configurations [2, 3, 9, 16, 28, 29, 55, 58, 62, 78]. The focus of these works is to verify desired properties (e.g., reachability) given precise specifications through detailed modeling of diverse network protocols (e.g., BGP, OSPF), and scaling the verification to larger networks. Elastispec is complementary, and seeks to bridge existing real-world practice in enterprise networks with such verification techniques. In particular,

a key goal is to enable existing network verification methods when specifications are not complete and informal.

Intent-driven networking and LLMs. Synthesizing configurations from natural language intents is starting to receive attention. [22, 34, 36, 37, 46, 49, 72]. NetConfEval [72] develops a benchmark suite to evaluate the effectiveness of LLMs for network configuration tasks. CEGS [46] synthesizes configurations generalizing from examples with other topologies. [49] explores verification of LLM generated configurations (using tools such as [66]) to reduce human involvement in LLM-based synthesis. CAIP [38] improves LLM-based misconfiguration detection through context-aware prompting. More broadly, LLMs have been applied to other network management tasks (e.g., diagnosis) [32, 47, 73–75]. In contrast, Elastispec formalizes configuration management when specifications (partially present in vendor documents) are incomplete. While Elastispec uses LLMs, the focus is on designing agents to achieve higher accuracy for two specific tasks (i) translating vendor documents (capturing the hierarchy, choice and optionality); and (ii) inferring local mappings.

Specification mining. Elastispec’s ability to infer formal specifications is inspired by the line of work on specification mining, from classical PL-based approaches [6, 26] to recent LLM-based approaches [25, 64]. Given network configurations, Config2Spec [13] determines which of a set of properties (e.g., reachability, waypointing) hold under k failures. In contrast, our focus is on deriving specifications from vendor documents, and on deriving entity mappings from configurations. Further, traffic allowed by configurations may often be a configuration error which Elastispec identifies.

8 Conclusion

We have presented Elastispec, a first step towards formal management of enterprise firewall policy when specifications are informal and incomplete. As part of Elastispec, we have developed (i) a custom DSL that is sufficiently expressive to capture policies in vendor documents of popular enterprise applications; (ii) a multi-step consistency-preserving LLM approach that increases translation accuracies from 61% to 96%. (ii) an approach to map the DSL to a concrete network environment by correlating multiple enterprise data sources uncovering 2.6X more application instances than a baseline limited by imperfect inventories (iii) a methodology to audit configurations against partial specifications, generating compliance trees and identifying out-of-spec flows. Evaluations with real-world enterprise firewall configurations show that Elastispec enables comparative analysis across multiple application instances and identifies configuration errors confirmed by operators. Overall, Elastispec shows the feasibility of principled approaches to configuration management with informal and incomplete specifications. **Generative AI.** Generative AI tools were used for natural language document processing and application recognition.

Ethics Statement. This work raises no ethical issues.

Acknowledgments. We thank the anonymous reviewers and shepherd for their feedback which helped to improve the paper, and Ryan Beckett for pointing out connections to angelic verification. This work was funded in part by the National Science Foundation (NSF) award CCF-2319425.

References

- [1] Anubhavnidhi Abhashkumar, Aaron Gember-Jacobson, and Aditya Akella. 2020. AED: incrementally synthesizing policy-compliant and manageable configurations. In *Proceedings of the 16th International Conference on Emerging Networking EXperiments and Technologies* (Barcelona, Spain) (CoNEXT '20). Association for Computing Machinery, New York, NY, USA, 482–495. doi:10.1145/3386367.3431304
- [2] Anubhavnidhi Abhashkumar, Aaron Gember-Jacobson, and Aditya Akella. 2020. Tiramisu: Fast Multilayer Network Verification. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, Santa Clara, CA, 201–219. <https://www.usenix.org/conference/nsdi20/presentation/abhashkumar>
- [3] Bhavish Agarwal, Ranjita Bhagwan, Tathagata Das, Siddharth Eswaran, Venkata N Padmanabhan, and Geoffrey M Voelker. 2009. NetPrints: Diagnosing Home Network Misconfigurations Using Shared Knowledge.. In *NSDI*, Vol. 9. 349–364.
- [4] Ehab S Al-Shaer and Hazem H Hamed. 2004. Modeling and management of firewall policies. *IEEE Transactions on network and service management* 1, 1 (2004), 2–10.
- [5] Amazon Web Services, Inc. 2026. *Ensure internetwork traffic privacy in Amazon VPC*. https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Security.html.
- [6] Glenn Ammons, Rastislav Bodik, and James R Larus. 2002. Mining specifications. *ACM Sigplan Notices* 37, 1 (2002), 4–16.
- [7] Carolyn Jane Anderson, Nate Foster, Arjun Guha, Jean-Baptiste Jeannin, Dexter Kozen, Cole Schlesinger, and David Walker. 2014. NetKAT: semantic foundations for networks. In *Proceedings of the 41st ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) (POPL '14). Association for Computing Machinery, New York, NY, USA, 113–126. doi:10.1145/2535838.2535862
- [8] Carolyn Jane Anderson, Nate Foster, Arjun Guha, Jean-Baptiste Jeannin, Dexter Kozen, Cole Schlesinger, and David Walker. 2014. NetKAT: semantic foundations for networks. *SIGPLAN Not.* 49, 1 (Jan. 2014), 113–126. doi:10.1145/2578855.2535862
- [9] Ryan Beckett, Aarti Gupta, Ratul Mahajan, and David Walker. 2017. A General Approach to Network Configuration Verification. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication* (Los Angeles, CA, USA) (SIGCOMM '17). Association for Computing Machinery, New York, NY, USA, 155–168. doi:10.1145/3098822.3098834
- [10] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, and David Walker. 2016. Don't Mind the Gap: Bridging Network-wide Objectives and Device-level Configurations. In *Proceedings of the 2016 ACM SIGCOMM Conference* (Florianopolis, Brazil) (SIGCOMM '16). Association for Computing Machinery, New York, NY, USA, 328–341. doi:10.1145/2934872.2934909
- [11] Ryan Beckett, Ratul Mahajan, Todd Millstein, Jitendra Padhye, and David Walker. 2017. Network configuration synthesis with abstract topologies. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Barcelona, Spain) (PLDI 2017). Association for Computing Machinery, New York, NY, USA, 437–451. doi:10.1145/3062341.3062367
- [12] Theophilus Benson, Aditya Akella, and David A. Maltz. 2009. Mining policies from enterprise network configuration. In *Proceedings of the 9th ACM SIGCOMM Conference on Internet Measurement* (Chicago, Illinois, USA) (IMC '09). Association for Computing Machinery, New York, NY, USA, 136–142. doi:10.1145/1644893.1644909
- [13] Rudiger Birkner, Dana Drachslar-Cohen, Laurent Vanbever, and Martin Vechev. 2020. Config2Spec: Mining Network Specifications from Network Configurations. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, Santa Clara, CA, 969–984. <https://www.usenix.org/conference/nsdi20/presentation/birkner>
- [14] Rastislav Bodik, Satish Chandra, Joel Galenson, Doug Kimelman, Nicholas Tung, Shaon Barman, and Casey Rodarmor. 2010. Programming with angelic nondeterminism. In *Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (Madrid, Spain) (POPL '10). Association for Computing Machinery, New York, NY, USA, 339–352. doi:10.1145/1706299.1706339
- [15] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: programming protocol-independent packet processors. *SIGCOMM Comput. Commun. Rev.* 44, 3 (July 2014), 87–95. doi:10.1145/2656877.2656890
- [16] Daniele Brighenti, Guido Marchetto, Riccardo Sisto, Fulvio Valenza, and Jalollidin Yusupov. 2022. Automated firewall configuration in virtual networks. *IEEE Transactions on Dependable and Secure Computing* 20, 2 (2022), 1559–1576.
- [17] Broadcom. 2024. Configuring Firewall and Ports. https://docs.automic.com/documentation/webhelp/english/AIR/12.6/DOCU/12.6/AIR%20Guides/Content/Common/Security/Security_Hardening_AEFirewallPorts.htm
- [18] Cisco Systems, Inc. 2017. *Security Configuration Guide: Access Control Lists, Cisco IOS Release 15M&T*. https://www.cisco.com/c/en/us/td/docs/ios-xml/ios/sec_data_acl/configuration/15-mt/sec-data-acl-15-mt-book.html.
- [19] Cisco Systems, Inc. 2019. *Cisco ACL Policy Model Guide*. <https://www.cisco.com/c/en/us/td/docs/switches/datacenter/aci/apic/sw/policy-model-guide/b-Cisco-ACL-Policy-Model-Guide.html>.
- [20] Ankush Das, Shuvendu K. Lahiri, Akash Lal, and Yi Li. 2015. Angelic Verification: Precise Verification Modulo Unknowns. In *Computer Aided Verification*, Daniel Kroening and Corina S. Păsăreanu (Eds.). Springer International Publishing, Cham, 324–342.
- [21] Wenlong Ding, Jianqiang Li, Zhixiong Niu, Huangxun Chen, Yongqiang Xiong, and Hong Xu. 2025. Automating Conflict-Aware ACL Configurations with Natural Language Intents. *arXiv preprint arXiv:2508.17990* (2025).
- [22] Denis Donadel, Francesco Marchiori, Luca Pajola, and Mauro Conti. 2024. Can LLMs Understand Computer Networks? Towards a Virtual System Administrator. *arXiv preprint arXiv:2404.12689* (2024).
- [23] Ahmed El-Hassany, Petar Tsankov, Laurent Vanbever, and Martin Vechev. 2017. Network-Wide Configuration Synthesis. In *Computer Aided Verification*, Rupak Majumdar and Viktor Kunčák (Eds.). Springer International Publishing, Cham, 261–281.
- [24] Ahmed El-Hassany, Petar Tsankov, Laurent Vanbever, and Martin Vechev. 2018. NetComplete: Practical Network-Wide Configuration Synthesis with Autocompletion. In *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)*. USENIX Association, Renton, WA, 579–594. <https://www.usenix.org/conference/nsdi18/presentation/el-hassany>
- [25] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. 2024. Can Large Language Models Transform Natural Language Intent into Formal Method Postconditions? *Proc. ACM Softw. Eng.* 1, FSE, Article 84 (July 2024), 24 pages. doi:10.1145/3660791
- [26] Michael D Ernst, Jeff H Perkins, Philip J Guo, Stephen McCamant, Carlos Pacheco, Matthew S Tschantz, and Chen Xiao. 2007. The Daikon system for dynamic detection of likely invariants. *Science of computer programming* 69, 1-3 (2007), 35–45.
- [27] Esri. 2025. Ports used by ArcGIS GeoEvent Server. <https://enterprise.arcgis.com/en/geoevent/latest/install/windows/ports-used-by-geoevent.htm>
- [28] Seyed K Fayaz, Tushar Sharma, Ari Fogel, Ratul Mahajan, Todd Millstein, Vyas Sekar, and George Varghese. 2016. Efficient network reachability analysis using a succinct control plane representation. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. 217–232.
- [29] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. 2015. A general approach to network configuration analysis. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. 469–483.
- [30] Aaron Gember-Jacobson, Raajay Viswanathan, Aditya Akella, and Ratul Mahajan. 2016. Fast Control Plane Analysis Using an Abstract Representation. In *Proceedings of the 2016 ACM SIGCOMM Conference* (Florianopolis, Brazil) (SIGCOMM '16). Association for Computing Machinery, New York, NY, USA, 300–313. doi:10.1145/2934872.2934876
- [31] Google Cloud. 2026. *VPC firewall rules*. <https://cloud.google.com/firewall/docs/firewalls>.
- [32] Pouya Hamadani, Behnaz Arzani, Sadjad Fouladi, Siva Kesava Reddy Kakarla, Rodrigo Fonseca, Denizcan Billor, Ahmad Cheema, Edet Nkposong, and Ranveer Chandra. 2023. A Holistic View of AI-driven Network Incident Management. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*. 180–188.
- [33] Timothy L Hinrichs, Natasha S Gude, Martin Casado, John C Mitchell, and Scott Shenker. 2009. Practical declarative network management. In *Proceedings of the 1st ACM workshop on Research on enterprise networking*. 1–10.
- [34] Zied Ben Houidi and Dario Rossi. 2022. Neural language models for network configuration: Opportunities and reality check. *Computer Communications* 193 (2022), 118–125.
- [35] HP Development Company, L.P. 2026. HP Web Jetadmin - Ports. https://support.hp.com/us-en/document/ish_3007439-11769595-16
- [36] Beni Ifland, Elad Duani, Rubin Krief, Miro Ohana, Aviram Zilberman, Andres Murillo, Ofir Manor, Ortal Lavi, Hikichi Kenji, Asaf Shabta, et al. 2024. Genet: A multimodal llm-based co-pilot for network topology and configuration. *arXiv preprint arXiv:2407.08249* (2024).
- [37] Arthur S. Jacobs, Ricardo J. Pfitscher, Rafael H. Ribeiro, Ronaldo A. Ferreira, Lisandro Z. Granville, Walter Willinger, and Sanjay G. Rao. 2021. Hey, Lumi! Using Natural Language for Intent-Based Network Management. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 625–639. <https://www.usenix.org/conference/atc21/presentation/jacobs>
- [38] Xi Jiang, Aaron Gember-Jacobson, and Nick Feamster. 2024. CAIP: Detecting Router Misconfigurations with Context-Aware Iterative Prompting of LLMs. *arXiv preprint arXiv:2411.14283* (2024).
- [39] Johnson Controls. 2023. *C•CURE 9000 and iSTAR Port Assignments*. <https://www.johnsoncontrols.com/-/media/project/jci-global/johnson-controls/us-region/united-states-johnson-controls/cyber-solutions/documents/ccure-9000-and-istar-port-assignments-v30-rev-d.pdf>
- [40] Juniper Networks, Inc. 2026. *Routing Policies, Firewall Filters, and Traffic Policers User Guide*. <https://www.juniper.net/documentation/us/en/software/junos/>

- routing-policy/.
- [41] Nanxi Kang, Ori Rottenstreich, Sanjay Rao, and Jennifer Rexford. 2015. Alpaca: compact network policies with attribute-carrying addresses. In *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies* (Heidelberg, Germany) (CoNEXT '15). Association for Computing Machinery, New York, NY, USA, Article 7, 13 pages. doi:10.1145/2716281.2836092
 - [42] Peyman Kazemian, George Varghese, and Nick McKeown. 2012. Header Space Analysis: Static Checking for Networks. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*. USENIX Association, San Jose, CA, 113–126. <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/kazemian>
 - [43] Ahmed Khurshid, Xuan Zou, Wenxuan Zhou, Matthew Caesar, and P. Brighten Godfrey. 2013. VeriFlow: Verifying Network-Wide Invariants in Real Time. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. USENIX Association, Lombard, IL, 15–27. <https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/khurshid>
 - [44] L-Soft Sweden AB. 2015. *LISTSERV Maestro Admin Tech Doc 7: Installing Behind a Firewall or Proxy*. https://www.ietf.org/standards/maestro/8.1/LMA_AdminTechDoc7_InstallingBehindAFirewallOrProxy.pdf
 - [45] Shuvendu K. Lahiri. 2024. Evaluating LLM-driven User-Intent Formalization for Verification-Aware Languages. In *2024 Formal Methods in Computer-Aided Design (FMCAD)*. 142–147. doi:10.34727/2024/isbn.978-3-85448-065-5_19
 - [46] Jianmin Liu, Li Chen, Dan Li, and Yukai Miao. 2025. CEGS: Configuration Example Generalizing Synthesizer. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*. USENIX Association, Philadelphia, PA, 1327–1347. <https://www.usenix.org/conference/nsdi25/presentation/liu-jianmin>
 - [47] Sathya Kumar Mani, Yajie Zhou, Kevin Hsieh, Santiago Segarra, Trevor Eberl, Eliran Azulai, Ido Frizler, Ranveer Chandra, and Srikanth Kandula. 2023. Enhancing network management using code generated by large language models. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*. 196–204.
 - [48] Microsoft Corporation. 2025. *Azure network security groups overview*. <https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview>
 - [49] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, and George Varghese. 2023. What do LLMs need to Synthesize Correct Router Configurations?. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks*. 189–195.
 - [50] Christopher Monsanto, Joshua Reich, Nate Foster, Jennifer Rexford, and David Walker. 2013. Composing software defined networks. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. 1–13.
 - [51] OCLC. 2025. How many ports does EZproxy use with Proxy by host-name? https://help.oclc.org/Library_Management/EZproxy/Get_started/How_many_ports_does_EZproxy_use
 - [52] Paessler GmbH. 2026. PRTG Manual: Default Ports. https://www.paessler.com/manuals/prtg/list_of_default_ports
 - [53] Palo Alto Networks, Inc. 2025. PAN-OS. <https://docs.paloaltonetworks.com/pan-os>
 - [54] PaperCut. 2025. Firewall Ports used by NG & MF. <https://www.papercut.com/kb/Main/FirewallPorts/>
 - [55] Santhosh Prabhu, Kuan Yen Chou, Ali Kheradmand, Brighten Godfrey, and Matthew Caesar. 2020. Plankton: Scalable network configuration verification through model checking. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. USENIX Association, Santa Clara, CA, 953–967. <https://www.usenix.org/conference/nsdi20/presentation/prabhu>
 - [56] Chaitan Prakash, Jeongkeun Lee, Yoshio Turner, Joon-Myung Kang, Aditya Akella, Sujata Banerjee, Charles Clark, Yadi Ma, Puneet Sharma, and Ying Zhang. 2015. PGA: Using Graphs to Express and Automatically Reconcile Network Policies. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication* (London, United Kingdom) (SIGCOMM '15). Association for Computing Machinery, New York, NY, USA, 29–42. doi:10.1145/2785956.2787506
 - [57] Sivaramakrishnan Ramanathan, Ying Zhang, Mohab Gawish, Yogesh Mundada, Zhaodong Wang, Sangki Yun, Eric Lippert, Walid Taha, Minlan Yu, and Jelena Mirkovic. 2023. Practical Intent-driven Routing Configuration Synthesis. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 629–644. <https://www.usenix.org/conference/nsdi23/presentation/ramanathan>
 - [58] Ronald W Ritchey and Paul Ammann. 2000. Using model checking to analyze network vulnerabilities. In *Proceeding 2000 IEEE Symposium on Security and Privacy. S&P 2000*. IEEE, 156–165.
 - [59] Kristin Yvonne Rozier. 2016. Specification: The biggest bottleneck in formal methods and autonomy. In *Working Conference on Verified Software: Theories, Tools, and Experiments*. Springer, 8–26.
 - [60] Sassafras Software Inc. 2020. Firewall Settings. <https://www.sassafras.com/software/release77/current/Documentation/Help/firewall.html>
 - [61] Tibor Schneider, Rüdiger Birkner, and Laurent Vanbever. 2021. Snowcap: synthesizing network-wide configuration updates. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Virtual Event, USA) (SIGCOMM '21). Association for Computing Machinery, New York, NY, USA, 33–49. doi:10.1145/3452296.3472915
 - [62] Samuel Steffen, Timon Gehr, Petar Tsankov, Laurent Vanbever, and Martin Vechev. 2020. Probabilistic Verification of Network Configurations. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication* (Virtual Event, USA) (SIGCOMM '20). Association for Computing Machinery, New York, NY, USA, 750–764. doi:10.1145/3387514.3405900
 - [63] Kausik Subramanian, Loris D'Antoni, and Aditya Akella. 2018. Synthesis of Fault-Tolerant Distributed Router Configurations. *Proc. ACM Meas. Anal. Comput. Syst.* 2, 1, Article 22 (April 2018), 26 pages. doi:10.1145/3179425
 - [64] Chuyue Sun, Viraj Agashe, Saikat Chakraborty, Jubi Taneja, Clark Barrett, David Dill, Xiaokang Qiu, and Shuvendu K. Lahiri. 2025. ClassInvGen: Class Invariant Synthesis using Large Language Models. In *AI Verification*, Gagandeep Singh and Caterina Urban (Eds.). Springer Nature Switzerland, Cham.
 - [65] Alan Tang, Ryan Beckett, Steven Benaloh, Karthick Jayaraman, Tejas Patil, Todd Millstein, and George Varghese. 2023. Lightyear: Using Modularity to Scale BGP Control Plane Verification. In *Proceedings of the ACM SIGCOMM 2023 Conference* (New York, NY, USA) (ACM SIGCOMM '23). Association for Computing Machinery, New York, NY, USA, 94–107. doi:10.1145/3603269.3604842
 - [66] Alan Tang, Siva Kesava Reddy Kakarla, Ryan Beckett, Ennan Zhai, Matt Brown, Todd Millstein, Yuval Tamir, and George Varghese. 2021. Campion: debugging router configuration differences. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference* (Virtual Event, USA) (SIGCOMM '21). Association for Computing Machinery, New York, NY, USA, 748–761. doi:10.1145/3452296.3472925
 - [67] Bingchuan Tian, Xinyi Zhang, Ennan Zhai, Hongqiang Harry Liu, Qiaobo Ye, Chunsheng Wang, Xin Wu, Zhiming Ji, Yihong Sang, Ming Zhang, Da Yu, Chen Tian, Haitao Zheng, and Ben Y. Zhao. 2019. Safely and automatically updating in-network ACL configurations with intent language. In *Proceedings of the ACM Special Interest Group on Data Communication* (Beijing, China) (SIGCOMM '19). Association for Computing Machinery, New York, NY, USA, 214–226. doi:10.1145/3341302.3342088
 - [68] Bingchuan Tian, Xinyi Zhang, Ennan Zhai, Hongqiang Harry Liu, Qiaobo Ye, Chunsheng Wang, Xin Wu, Zhiming Ji, Yihong Sang, Ming Zhang, Da Yu, Chen Tian, Haitao Zheng, and Ben Y. Zhao. 2019. Safely and automatically updating in-network ACL configurations with intent language. In *Proceedings of the ACM Special Interest Group on Data Communication* (Beijing, China) (SIGCOMM '19). Association for Computing Machinery, New York, NY, USA, 214–226. doi:10.1145/3341302.3342088
 - [69] Tigera, Inc. 2026. *Calico Network Policy*. <https://docs.tigera.io/calico/latest/reference/resources/networkpolicy>
 - [70] Veeam. 2025. Veeam Agent Management Guide. https://helpcenter.veeam.com/archive/backup/120/agents/used_ports.html
 - [71] Anduo Wang, Limin Jia, Wencho Zhou, Yiqing Ren, Boon Thau Loo, Jennifer Rexford, Vivek Nigam, Andre Scedrov, and Carolyn Talcott. 2012. FSR: Formal Analysis and Implementation Toolkit for Safe Interdomain Routing. *IEEE/ACM Transactions on Networking* 20, 6 (2012), 1814–1827. doi:10.1109/TNET.2012.2187924
 - [72] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, and Marco Chiesa. 2024. NetConfEval: Can LLMs Facilitate Network Configuration? *Proceedings of the ACM on Networking* 2, CoNEXT2 (2024), 1–25.
 - [73] Chenxu Wang, Xumiao Zhang, Runwei Lu, Xianshang Lin, Xuan Zeng, Xinlei Zhang, Zhe An, Gongwei Wu, Jiaqi Gao, Chen Tian, Guilai Chen, Guyue Liu, Yuhong Liao, Tao Lin, Dennis Cai, and Ennan Zhai. 2025. Towards LLM-Based Failure Localization in Production-Scale Networks. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) (SIGCOMM '25). Association for Computing Machinery, New York, NY, USA, 496–511. doi:10.1145/3718958.3750505
 - [74] Haopei Wang, Anubhavnidhi Abhashkumar, Changyu Lin, Tianrong Zhang, Xiaoming Gu, Ning Ma, Chang Wu, Songlin Liu, Wei Zhou, Yongbin Dong, Weirong Jiang, and Yi Wang. 2024. NetAssistant: Dialogue Based Network Diagnosis in Data Center Networks. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 2011–2024. <https://www.usenix.org/conference/nsdi24/presentation/wang-haopei>
 - [75] Zhaodong Wang, Samuel Lin, Guanqing Yan, Soudeh Ghorbani, Minlan Yu, Jiawei Zhou, Nathan Hu, Lopa Baruah, Sam Peters, Srikanth Kamath, Jerry Yang, and Ying Zhang. 2025. Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework. In *Proceedings of the ACM SIGCOMM 2025 Conference* (São Francisco Convent, Coimbra, Portugal) (SIGCOMM '25). Association for Computing Machinery, New York, NY, USA, 347–362. doi:10.1145/2983990.2984012
 - [76] Konstantin Weitz, Doug Woos, Emina Torlak, Michael D. Ernst, Arvind Krishnamurthy, and Zachary Tatlock. 2016. Scalable verification of border gateway protocol configurations with an SMT solver. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications* (Amsterdam, Netherlands) (OOPSLA 2016). Association for Computing Machinery, New York, NY, USA, 765–780. doi:10.1145/2983990.2984012
 - [77] Xieyang Xu, Yifei Yuan, Zachary Kincaid, Arvind Krishnamurthy, Ratul Mahajan, David Walker, and Ennan Zhai. 2024. Relational Network Verification. In *Proceedings of the ACM SIGCOMM 2024 Conference* (Sydney, NSW, Australia)

- (ACM SIGCOMM '24). Association for Computing Machinery, New York, NY, USA, 213–227. doi:10.1145/3651890.3672238
- [78] Fangdan Ye, Da Yu, Ennan Zhai, Hongqiang Harry Liu, Bingchuan Tian, Qiaobo Ye, Chunsheng Wang, Xin Wu, Tianchen Guo, Cheng Jin, Duncheng She, Qing Ma, Biao Cheng, Hui Xu, Ming Zhang, Zhiliang Wang, and Rodrigo Fonseca. 2020. Accuracy, Scalability, Coverage: A Practical Configuration Verifier on a Global WAN. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication* (Virtual Event, USA) (SIGCOMM '20). Association for Computing Machinery, New York, NY, USA, 599–614. doi:10.1145/3387514.3406217
- [79] Lihua Yuan, Hao Chen, Jianing Mai, Chen-Nee Chuah, Zhendong Su, and P. Mohapatra. 2006. FIREMAN: a toolkit for firewall modeling and analysis. In *2006 IEEE Symposium on Security and Privacy (S&P'06)*. 15 pp.–213. doi:10.1109/SP.2006.16

Appendices are supporting material that has not been peer-reviewed.

A Enriching Inventory by Uncovering Application (§4.2)

This appendix details the implementation of the inventory enrichment technique introduced in §4.2, aiming at resolving uninformative labels (e.g., Brownfield). We first describe the multi-stage querying design and then evaluate its accuracy in identifying applications from network port configurations.

A.1 Heuristic Design

As described in the main text, our goal is to identify the running application by analyzing the signature of ports found in the network configuration. The process proceeds as follows: given a list of <prot,port>pairs, the agent filters out common <prot,port>pairs (e.g., TCP/80, TCP/443) and queries an LLM to provide a minimal set of applications that together explain as many ports as possible. Unfortunately, direct queries of this form did not perform well (A.2). Instead the agent repeats the query multiple times, and aggregates as follows:

- The agent asks the LLM to normalize the result from each query to (i) group variants of the same application obtained during different runs (e.g., PaperCut MF, PaperCut MF/NG) under a single, canonical name (e.g., PaperCut); and (ii) filter out generic services (e.g., Remote Desktop Protocol).
- Next, the agent builds a matrix indicating which pairs are associated with each application. The agent does so by running specific queries that ask the LLM if each candidate application in the prior step is associated with a particular pair. We find this exhaustive checking improves accuracy as the LLM can focus on a single pair in this step.
- Next the agent decides the likely set of application(s) that cover the ports (note that the same server may host multiple applications). The agent does so by (i) assigning a score for each application based on which ports are relevant to it weighted by port uniqueness (ports are assigned higher weights if fewer candidate applications match indicating a more reliable signal); and (ii) using a greedy weighted set cover algorithm that covers most ports with the fewest applications terminating when the number of uncovered ports is small.

LLM selection. The above algorithm performed relatively well with both Gemini-2.5-Pro and GPT-5, but some applications were consistently missed by each of them (A.2) with coverage improved if both LLMs were queried. Consequently, the agent queried both LLMs and used the procedure above to merge the results. Further, the exhaustive matrix verification in the second step was expensive. Hence, we used two lighter weight LLM models (Gemini-2.5-Flash and GPT-5-Mini) for this step only consulting the heavier weight Gemini-2.5-Pro if the two did not agree.

A.2 Performance Evaluation

To validate the design of our agent (A.1), we isolate its performance against simpler LLM-based strategies. We assess the ability of different model architectures and algorithmic heuristics to correctly identify an application given only its associated protocol-port pairs.

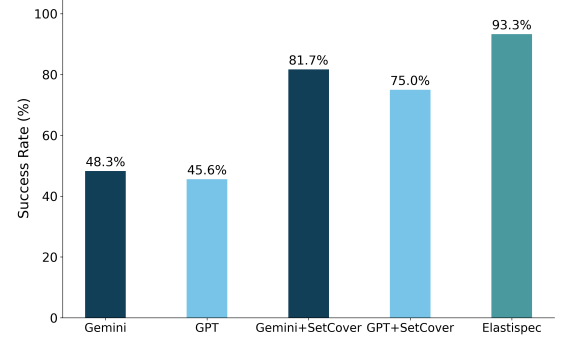


Figure 10: Performance Comparison of Service Inference Methodologies

We evaluated our methodology on 20 groups of protocol–port pairs where ground truth was available.

We compare Elastispec’s approach with several baselines: (i) *Gemini* and *GPT* represent schemes that simply ask Gemini-2.5-Pro and GPT-5 to provide the most relevant application(s); (ii) *Gemini+SetCover*, and *GPT+SetCover* query each LLM for candidate applications, and the relevant ports for each application. A weighted set cover algorithm (A.1) is then run to identify the likely applications. (iii) *Elastispec* represents the approach in Appendix A.1 where both LLMs are queried for candidate applications, an explicit step verifying if an application uses a particular port is done for all combinations of candidate applications and ports, and finally a weighted set cover algorithm is used to make the final decision.

All LLMs use default parameters with web search enabled across model and method. We evaluate with top-1 accuracy, where the ground truth application must be the highest confidence candidate per protocol–port group.

Figure 10 presents the comparison results. The results show that (i) directly asking the LLMs is insufficient with Gemini-2.5-Pro and GPT-5 only achieving accuracies of 48.3% and 45.6% respectively. (ii) combining each LLM with Elastispec’s weighted set cover algorithm improves accuracies to 81.7% and 75% for the two LLMs; Interestingly, some applications are consistently missed by each of the LLMs. For example, Gemini-2.5-Pro often misses LSoft, while GPT-5 tends to miss CCure. These model-specific blind spots underscore the need to consult both models in our final design. (iii) *Elastispec* achieves accuracies of 93.3% indicating its efficacy and the necessity of combining multi LLMs. Overall, the results show the feasibility of using LLMs to identify mappings from logical entities to the physical topology.

B Elastispec’s Verification Algorithm

We now present Elastispec’s verification algorithm. Recall that our verification yields one of three results: VAL ID, UNSAT, and CONJECTURED–SAT. A Elastispec specification has a hierarchical tree structure, and Elastispec verifies it in a bottom-up manner, starting from the leaves of the tree representing atomic connectivity policies and moving upward along the tree, and assigns a label for each node of the tree. Below we discuss different aspects of the algorithm:

Algorithm 1 TreeVerify($\mathcal{N}, M, \mathcal{I}, t$)

Require: Network model $\mathcal{N}$, base mapping M , interfaces/subnets $\mathcal{I}$, subtree t

Ensure: (v, Φ) where $v \in \{\text{VALID}, \text{UNSAT}, \text{CONJECTURED-SAT}\}$ and Φ is a symbolic conjecture formula

```

1: if  $t$  is a leaf  $\ell$  then
2:   return LEAFVERIFY( $\mathcal{N}, M, \mathcal{I}, \ell$ )
3: end if
4:  $ChildResults \leftarrow \emptyset$ 
5: for each child  $c$  of  $t$  do
6:   if  $c$  is optional then
7:     TREEVERIFY( $\mathcal{N}, M, \mathcal{I}, c$ )            $\triangleright$  for reporting only
8:   else
9:     append TREEVERIFY( $\mathcal{N}, M, \mathcal{I}, c$ ) to  $ChildResults$ 
10:  end if
11: end for
12: return COMPOSE(op( $t$ ),  $ChildResults$ )

```

the oracle it leverages the leaf-level verification process, and the upward propagation process.

Oracle. We leverage Batfish [29] as a reachability oracle. For a fully mapped connectivity query, Elastispec uses Batfish in the standard way: it checks whether a reachable witness flow exists, and whether a counterexample exists showing that the requirement is only partially satisfied. For a query with exactly one unresolved endpoint, Elastispec issues a wildcard reachability query in which that endpoint is left unconstrained (i.e., “*”). We then unbox Batfish’s internal representation and extract the BDD for the satisfying packet space. Elastispec projects this BDD onto the unresolved endpoint IP space and uses the result as the conjecture space for that role. Thus, rather than enumerating candidate endpoints or testing representative flows, Elastispec derives all satisfying IP assignments for the unresolved role at once.

For an atomic connectivity policy, the verification output is determined as follows:

-
- **VALID** if both endpoint roles are mapped and the requirement is fully satisfied for the relevant endpoint sets under the current mapping. Concretely, Batfish finds at least one reachable witness flow satisfying the leaf, and no counterexample showing partial satisfaction.
-
- **UNSAT** if the requirement is not satisfied, is only partially satisfied, or if both endpoint roles are unmapped so the requirement cannot be grounded to concrete endpoints. In the fully mapped case, this includes either the absence of any reachable witness flow, or the presence of a Batfish counterexample showing that the requirement does not hold for all relevant endpoint pairs.
-
- **CONJECTURED-SAT** if the requirement becomes satisfiable when exactly one endpoint role is left unresolved. In this case, Elastispec uses Batfish’s internal representation and returns a BDD over that role’s IP space representing all assignments that satisfy the leaf.

Algorithm 1: TreeVerify

Algorithm 2: LeafVerify (simplified dispatcher)

Algorithm 2 LeafVerify($\mathcal{N}, M, \mathcal{I}, \ell$)

Require: Network model $\mathcal{N}$, base mapping M , interfaces/subnets $\mathcal{I}$, leaf ℓ

Ensure: (v, Φ)

```

1: if both endpoints of  $\ell$  are mapped then
2:   return VERIFYMAPPELEAF( $\mathcal{N}, M, \ell$ )
3: else if both endpoints of  $\ell$  are unmapped then
4:   return (UNSAT,  $\perp$ )
5: else
6:   return VERIFYLEAFWITHONEUNMAPPELEAF( $\mathcal{N}, M, \mathcal{I}, \ell$ )
7: end if

```

Algorithm 3: VerifyMappedLeaf (ReachExists/HasCounterexample)

Algorithm 3 VerifyMappedLeaf($\mathcal{N}, M, \ell$)

Require: Network model $\mathcal{N}$, base mapping M , mapped leaf ℓ

Ensure: (v, Φ)

```

1: if REACH EXISTS( $\mathcal{N}, M, \ell$ ) = false then
2:   return (UNSAT,  $\perp$ )
3: else if HAS COUNTEREXAMPLE( $\mathcal{N}, M, \ell$ ) = true then
4:   return (UNSAT,  $\perp$ )
5: else
6:   return (VALID,  $\top$ )
7: end if

```

Algorithm 4: VerifyLeafWithOneUnmapped (BDD-based)

Algorithm 4 VerifyLeafWithOneUnmapped($\mathcal{N}, M, \mathcal{I}, \ell$)

Require: Network model $\mathcal{N}$, base mapping M , interfaces/subnets $\mathcal{I}$, leaf ℓ

Ensure: (v, Φ)

```

1:  $r_u \leftarrow$  the unmapped role in  $\ell$ 
2:  $B \leftarrow$  BUILDREACHABLEBDD( $\mathcal{N}, M, \mathcal{I}, \ell, r_u$ )
3: if  $B = \emptyset$  then
4:   return (UNSAT,  $\perp$ )
5: else
6:   return (CONJECTURED-SAT, ( $r_u \in B$ ))
7: end if

```

Leaf verdicts. Algorithm 2 dispatches leaf verification based on how many endpoint roles are mapped. If both endpoints are mapped, Algorithm 3 checks whether the required connectivity is fully satisfied under the current mapping. If both endpoints are unmapped, the leaf is reported as UNSAT, since the requirement cannot be grounded to concrete endpoints. If exactly one endpoint is unmapped, Algorithm 4 computes a BDD over that role’s IP space representing all assignments that satisfy the leaf. The leaf is CONJECTURED-SAT iff this BDD is non-empty.

Composition of child results. COMPOSE in Algorithm 1 combines the results returned by the required children of an internal node according to the node operator. Each child result is a pair (v_i, Φ_i) , where $\Phi_i = \top$ for VALID, $\Phi_i = \perp$ for UNSAT, and otherwise Φ_i is a symbolic conjecture formula. For a conjunction, all required children must hold, so COMPOSE forms $\Phi = \bigwedge_i \Phi_i$; if any child is UNSAT, this simplifies to $\perp$. For a disjunction, it suffices that one required child hold, so COMPOSE forms $\Phi = \bigvee_i \Phi_i$ after discarding UNSAT children; if no required child remains, the result is $\perp$. The composed formula is then normalized: constraints over the same unmapped role are combined by BDD intersection under conjunction and by BDD union under disjunction.

Tree satisfaction. Algorithm 1 evaluates the specification tree bottom-up. For each internal node, it recursively verifies the required children, ignores optional children when determining parent satisfiability, and combines child results using COMPOSE. After normalization, if the composed formula simplifies to $\perp$, the subtree is reported as UNSAT; if it simplifies to $\top$, the subtree is reported as VALID; otherwise, it is reported as CONJECTURED-SAT.

Search space construction. BUILDREACHABLEBDD is used when exactly one endpoint of a leaf is mapped. It uses the mapped-side IPs together with the service constraint (protocol π , destination port P) to identify the firewall interface(s) for the mapped subnet and the ACL rule(s) relevant to the leaf. Elastispec then issues a Batfish reachability query in which the unmapped endpoint is left unconstrained (i.e., “”). Batfish computes the satisfying packet space and internally represents it as a BDD. Elastispec projects this BDD onto the unmapped endpoint IP space and uses the result as the conjecture for that role. Thus, Elastispec derives the full satisfying assignment space for the unmapped role in one step.

C Prompts for Translating Vendor Docs to Elastispec (§4.1)

This appendix contains the prompts for translating vendor docs to Elastispec, organized according to the workflow described in the main paper. The prompts shown in this section have been condensed for brevity; the full prompts include more detailed instructions and in-context examples.

(A1) Outline Generator. The workflow begins with this agent, which provides the global structure for the entire process. It processes a technical document to construct a hierarchical JSON outline resembling a table of contents. This structured outline is crucial for guiding the subsequent extraction and generation steps. The agent is specifically designed to parse complex structures, such as tables detailing network flows with “From” and “To” columns, into well-defined sections, as shown in Listing 3.

```

1 Create a structured, hierarchical outline (like a table of contents)
  from a document.
2
3 **SPECIAL LOGIC FOR TABLES:**
4 For data tables with "From" and "To" columns, create nested
  subsections for each row formatted as "FROM -> TO".
5 =====
6 For example:
7
```

```

1 Your sole purpose is to extract text from a document based on a
  given path.
2
3 **CRITICAL RULES:** Your output MUST contain only the requested text
4
5 **INSTRUCTIONS:** Based on the **Current Section Path**, extract the
  corresponding text.
6 =====
7 For example:
8
9 INPUT (Current Section Path):
10 "Application Server - User Web Interface"
11
12 OUTPUT (Extracted Content):
13 Users access Application via web browser to release print jobs...
14 - TCP Port 443 (HTTPS)
15 ...

```

Listing 4: Content Extractor Prompt

```

8 INPUT (Document URL):
9 https://www.papercut.com/kb/Main/FirewallPorts/
10
11 OUTPUT (Generated Outline):
12 {
13   "Application Server": { "User Web Interface": null, ... },
14   "Ports for client comms": { "Client to server": null },
15   ...
16 }

```

Listing 3: Outline Generator Prompt

(A2) Content Extractor. Once the document is structured, this agent extracts the full text for a specific section from the original document based on a given outline path. It operates with strict rules to ensure that only the relevant content is retrieved, preventing the inclusion of extraneous information and providing clean input for the policy extraction agent, as illustrated in Listing 4.

(B) Flow Generator. This agent analyzes a section of documentation to extract key security policy details. It identifies firewall rules, network entities (e.g., “PaperCut Server”), and groups, tagging each policy with a unique ID and its active condition (e.g., “when using an Oracle database”). It uses a canonical model to avoid duplicating information, ensuring a clean, flat list of policies as shown in Listing 5.

```

1 Analyze a documentation section and extract structured data.
2
3 **INSTRUCTIONS:**
4 Extract **new** firewall policies, entities, and groups not in the
  canonical model. **Do NOT duplicate** any existing information.
5 =====
6 For example:
7
8 INPUT (Content for this Section):
9 The Application Server communicates with the Site Server over port
  1234 (TCP).
10
11 OUTPUT (Extracted Data):
12 {
13   "policies": [{

```

```

14     "policy_id": "App_to_Site", "source": "App Server", "dest": "
15       Site Server",
16       "port": ["1234"], "protocol": ["TCP"]
17     },
18     "entities": [{"name": "Site Server", ...}],
19     "groups": {}
20 }

```

Listing 5: Leaf Policy Flow Generator Prompt

(C1) *Leaf Logic and Optionality Generator.* For each individual feature, this agent defines its internal logic by creating a "leaf policy." It aggregates the relevant policy IDs and defines their relationship using logical operators (AND, OR), while also making an initial assessment of whether each constituent policy is optional for the feature based on the documentation, as shown in Listing 6.

```

1 Define the firewall policies for a single, specific feature.
2
3 **INSTRUCTIONS:**
4 1. **`operands`**: List all `policy_id`s for this feature.
5 2. **`logic`**: Define the relationship between them ("AND", "OR",
6    etc.).
7 3. **`is_optional`**: Determine if a `policy_id` is optional for
8    the entire feature.
9 =====
10 For example:
11
12 INPUT (Policies for Feature "LDAP"):
13 [
14   {"policy_id": "LDAP_Standard", ...},
15   {"policy_id": "LDAPS_Encrypted", ...}
16 ]
17
18 OUTPUT (Generated Policy):
19 {
20   "LDAP_Policy": {
21     "operands": [
22       {"policy_id": "LDAP_Standard", "is_optional": false},
23       {"policy_id": "LDAPS_Encrypted", "is_optional": false}
24     ],
25     "logic": "LDAP_Standard OR LDAPS_Encrypted"
26   }
27 }

```

Listing 6: Leaf Logic Generator Prompt

(C2) *Resolve Logical Dependencies Between Policy Nodes.* This agent addresses the challenge of resolving conflicts between different features. To ensure consistency, it identifies coupled policies (e.g., features that depend on a shared choice like "SDK1") and refactors them to enforce coherent configurations, as shown in the example in Listing 7.

```

1 Identify and resolve logical dependencies between feature policies.
2
3 **INSTRUCTIONS:**
4 1. **Identify Coupled Groups:** Find policies coupled by a shared
5    choice.
6 2. **Create Wrapper Policies:** Create a new wrapper policy to
7    manage their logic.
8 3. **Restructure Policies:** Break down original policies into
9    granular components.

```

```

7 4. **Output:** Respond with a JSON object containing: `
8    newly_created_policies`, `policies_to_remove`, and `
9    path_updates`.
10 =====
11 For example:
12
13 INPUT (Flawed policies):
14 { "P_Inbound": { "logic": "(P_A_SDK1 OR P_B_SDK2)" }, ... }
15
16 OUTPUT (Plan to enforce consistent choice):
17 {
18   "newly_created_policies": {
19     "Unified_SDK_Policy": { "logic": "SDK1_Policy OR SDK2_Policy" },
20     "SDK1_Policy": { "logic": "P_A_SDK1 AND P_C_SDK1" }
21   },
22   "policies_to_remove": ["P_Inbound", "P_Outbound"], ...
23 }

```

Listing 7: Resolve Logical Dependencies Between Policy Nodes Prompt

(D1) *Hierarchical Specification Generator.* Using the annotated outline from the first step, this agent constructs the parent nodes of the Specification hierarchy. It defines the logical relationships (AND, OR) between sibling nodes, thereby building up the overall structure of the application's firewall policies from the bottom up, as illustrated in Listing 8.

```

1 Create the high-level structure of a firewall policy from an "
2   annotated outline".
3
4 **INSTRUCTIONS:**
5 1. **Create Parent Policies:** For each key with a nested object,
6    create a parent policy.
7 2. **Define Logic:** Define the `logic` for the parent policy's `
8    operands`.
9 =====
10 For example:
11
12 INPUT (Annotated Outline):
13 { "App_A": { "User_Dir": { "Google": "G_Policy", "Azure": "A_Policy"
14   }, ... } }
15
16 OUTPUT (Parent Policies):
17 {
18   "App_A_Policy": { "logic": "User_Dir_Policy AND MFD_Policy", ...
19   },
20   "User_Dir_Policy": { "logic": "G_Policy OR A_Policy", ... }
21 }

```

Listing 8: Hierarchical Policy Generator Prompt

(D2) *Optionality Agent.* As a final step, this agent performs a global analysis of the entire specification hierarchy to make an authoritative decision on the optional status for every policy. Its core principle is that a policy's optionality is relative to its parent, a rule it applies recursively to ensure the final specification is valid, as shown in Listing 9.

```

1 Determine the optionality of each feature (policy) in the hierarchy.
2
3 **Core Principle:** A policy's `is_optional` flag is relative to its
4   parent.
5
6 **Crucial Rule for OR Logic:**

```

```
5 If `P.logic = "A OR B OR C"`, then `A`, `B`, and `C` must have `
  is_optional: false`.
6 =====
7 For example:
8
9 INPUT (A hierarchy with a required choice):
10 {
11   "AppPolicy": { "logic": "UserDir_Policy AND DB_Policy" },
12   "UserDir_Policy": { "logic": "Google_Policy OR Azure_Policy" },
13   ...
14 }
```

```
14
15 OUTPUT (Determined optionality for each policy):
16 {
17   "AppPolicy": false, "UserDir_Policy": false, "Google_Policy":
18     false, ...
19 }
```

Listing 9: Optionality Agent Prompt